UNIVERSITATEA DIN BUCUREȘTI
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ

PROIECT

cu tema:

„Magazin online”

Nume și prenume student: Miu Georgian-Fabian
Coordonator: Lect. dr. Silviu Laurențiu Vasile
Materie: Sisteme de gestiune a bazelor de date
Seria: 24
Grupa: 241
Anul universitar 2025-2026

Contents

Introducere 3

1. Prezentarea pe scurt a bazei de date (utilitatea ei) 4

2. Diagrama entitate-relație (ERD) 6

3. Diagrama conceptuală 7

4. Definirea tabelelor, adăugând toate constrângerile de integritate necesare (chei primare, chei externe etc.) 8

5. Adăugarea de informații coerente în tabelele create (minimum 5 înregistrări pentru fiecare entitate independentă; minimum 10 înregistrări pentru fiecare tabel asociativ) 24

6. Formularea în limbaj natural a unei probleme pe care să o rezolvăm folosind un subprogram stocat independent care să utilizeze toate cele 3 tipuri de colecții studiate și apelarea acestuia 35

7. Formularea în limbaj natural a unei probleme pe care să o rezolvăm folosind un subprogram stocat independent care să utilizeze 2 tipuri diferite de cursoare studiate, unul dintre acestea fiind cursor parametrizat, dependent de celălalt cursor și apelarea acestuia 39

8. Formularea în limbaj natural a unei probleme pe care să o rezolvăm folosind un subprogram stocat independent de tip funcție care să utilizeze într-o singură comandă SQL 3 dintre tabelele create. Vom trata toate excepțiile care pot apărea, incluzând excepțiile predefinite NO_DATA_FOUND și TOO_MANY_ROWS. Vom apela subprogramul astfel încât vom evidenția toate cazurile tratate 43

9. Formularea în limbaj natural a unei probleme pe care să o rezolvăm folosind un subprogram stocat independent de tip procedură care să aibă minimum 2 parametri și să utilizeze într-o singură comandă SQL 5 dintre tabelele create. Vom defini minimum 2 excepții proprii, altele decât cele predefinite la nivel de sistem. Vom apela subprogramul astfel încât vom evidenția toate cazurile definite și tratate. 47

10. Definirea unui trigger de tip LMD la nivel de comandă și declanșarea acestuia 51

11. Definirea unui trigger de tip LMD la nivel de linie și declanșarea acestuia 53

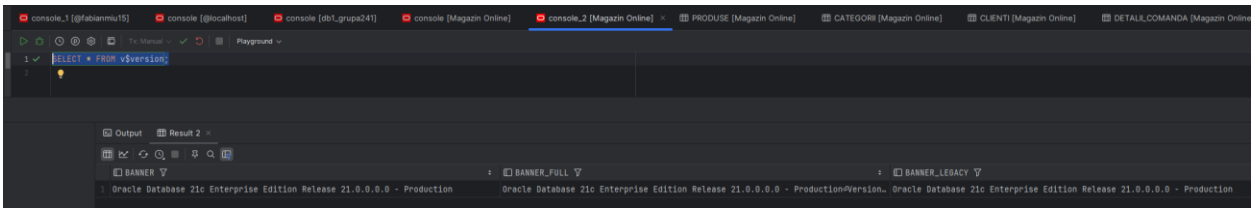
12. Definirea unui trigger de tip LDD și declanșarea acestuia 56

13. Formularea în limbaj natural a unei probleme pe care să o rezolvăm folosind un pachet care să includă tipuri de date complexe și obiecte necesare unui flux de acțiuni integrate, specifice bazei de date definite (minimum 2 tipuri de date, minimum 2 funcții, minimum 2 proceduri) 58

Introducere

Tema aleasă pentru proiectul la **Sisteme de gestiune a bazelor de date** este **Magazin Online**, așa cum va fi detaliat mai jos.

Versiunea SGBD-ului este **Oracle Database 21c Enterprise Edition (Release 21.0.0.0.0)**, instalat și rulat **direct pe sistemul local**, fără utilizarea unei mașini virtuale:



Aplicația a fost dezvoltată pe sistemul de operare **Windows 11**.

1. Prezentarea pe scurt a bazei de date (utilitatea ei)

Modelul propus reflectă funcționarea unui magazin online (similar cu eMAG) care permite clienților să achiziționeze produse de la diverși furnizori, din diverse categorii și să urmărească comenzile plasate.

Sistemul permite automatizarea și digitalizarea activităților esențiale într-un magazin online:

- Gestionarea produselor și a stocurilor acestora
- Urmărirea comenzilor și a procesării acestora
- Managementul relației cu angajații, furnizorii și clienții
- Generarea automată de facturi

Entități

1. CLIENTI
Entitatea stochează informațiile despre clienții magazinului online.
Fiecare client se poate înregistra în sistem și poate plasa una sau mai multe comenzi.

2. ANGAJATI
Entitatea reține datele angajaților magazinului.
Angajații sunt responsabili de procesarea comenzilor plasate de clienți.

3. PRODUSE
Entitatea conține informațiile despre produsele comercializate în magazin.

4. CATEGORII
Entitatea stochează categoriile de produse disponibile în magazin.

5. FURNIZORI
Entitatea conține informațiile despre furnizorii produselor.

6. COMENZI
Entitatea stochează informațiile despre comenzile plasate de clienți.

7. FACTURI
Entitatea reține datele despre facturile emise pentru comenzile plasate.

Relații

- **CLIENTI (1) — COMENZI (M)**
  Un client poate plasa mai multe comenzi sau poate să nu plaseze niciuna (doar să navigheze pe site), iar o comandă este plasată de un singur client.

- **COMENZI (M) — PRODUSE (M)**, prin tabelul asociativ **DETALII_COMANDA**
  O comandă poate conține mai multe produse sau niciunul, iar un produs poate apărea în mai multe comenzi sau în niciuna.

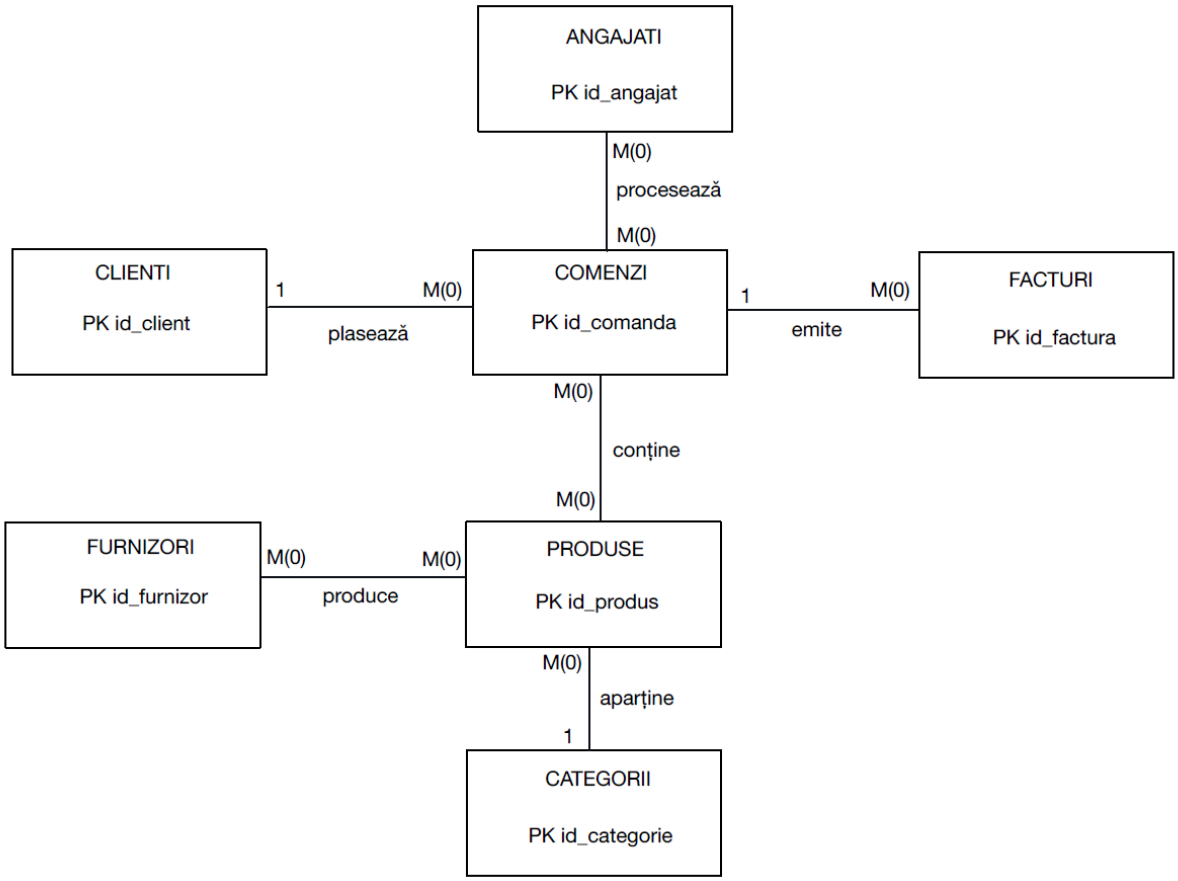
- **ANGAJATI (M) — COMENZI (M)**, prin tabelul asociativ **PROCESARE_COMENZI**
  Un angajat poate procesa mai multe comenzi sau niciuna (poate fi în concediu, spre exemplu), iar o comandă poate fi procesată de mai mulți angajați sau de niciunul (se poate procesa și singură).

- **COMENZI (1) — FACTURI (M)**
  O comandă poate emite mai multe facturi (proforma, factura finală etc.) sau niciuna (comenzile nefinalizate, anulate, respinse etc.), iar o factură este emisă pentru o singură comandă.

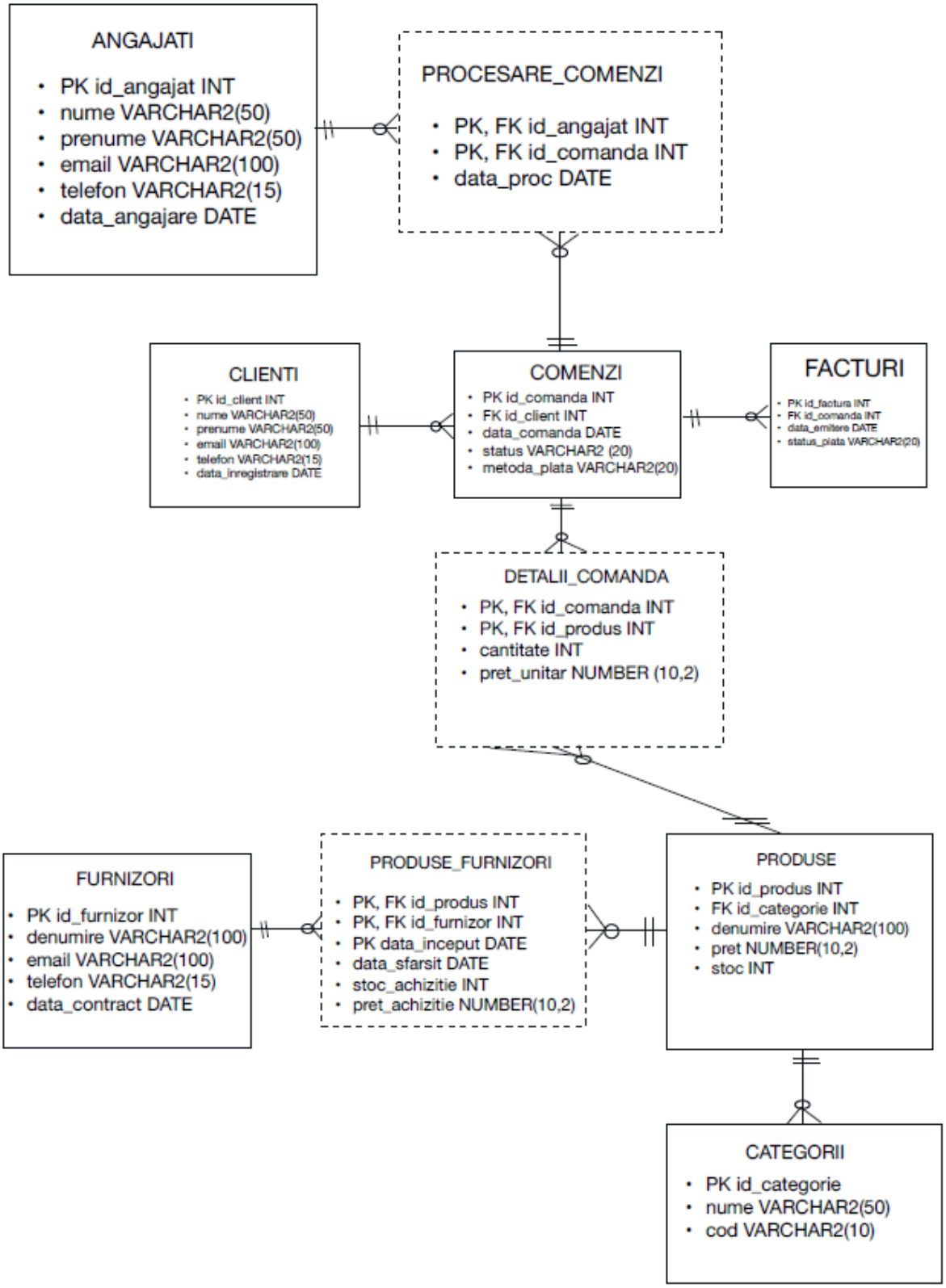
- **CATEGORII (1) — PRODUSE (M)**
  O categorie poate conține mai multe produse sau niciunul (spre exemplu, în cazul stocului epuizat), iar un produs aparține unei singure categorii.

- **FURNIZORI (M) — PRODUSE (M)**, prin tabelul asociativ **PRODUSE_FURNIZORI**
  Un furnizor poate produce mai multe produse sau niciunul (furnizor nou adăugat, inactiv temporar, arhivat etc.), iar un produs poate fi furnizat de mai mulți furnizori.

2. Diagrama entitate-relație (ERD)



3. Diagrama conceptuală



4. **Definirea tabelelor, adăugând toate constrângerile de integritate necesare (chei primare, chei externe etc.)**

```
CREATE TABLE CLIENTI (
   id_client          INT,
   nume               VARCHAR2(50) NOT NULL,
   prenume            VARCHAR2(50) NOT NULL,
   email              VARCHAR2(100) NOT NULL,
   telefon            VARCHAR2(15),
   data_inregistrare   DATE DEFAULT SYSDATE NOT NULL,
   CONSTRAINT pk_clienti PRIMARY KEY (id_client),
   CONSTRAINT uq_clienti_email UNIQUE (email),
   CONSTRAINT ck_clienti_email CHECK (email LIKE '%@%.%')
);


CREATE TABLE ANGAJATI (
   id_angajat         INT,
   nume               VARCHAR2(50) NOT NULL,
   prenume            VARCHAR2(50) NOT NULL,
   email              VARCHAR2(100) NOT NULL,
   telefon            VARCHAR2(15),
   data_angajare       DATE DEFAULT SYSDATE NOT NULL,
   CONSTRAINT pk_angajati PRIMARY KEY (id_angajat),
   CONSTRAINT uq_angajati_email UNIQUE (email),
   CONSTRAINT ck_angajati_email CHECK (email LIKE '%@%.%')
);

CREATE OR REPLACE TRIGGER trg_check_data_angajare -- cerinta 11, declansat de INSERT-urile
din ANGAJATI
BEFORE INSERT OR UPDATE ON ANGAJATI
FOR EACH ROW
BEGIN
   IF :NEW.data_angajare > SYSDATE THEN
      RAISE_APPLICATION_ERROR(-20001, 'Data angajarii nu poate fi in viitor!');
   END IF;
END;
/

CREATE TABLE CATEGORII (
   id_categorie        INT,
   nume               VARCHAR2(50) NOT NULL,
   cod                VARCHAR2(10) NOT NULL,
   CONSTRAINT pk_categorii PRIMARY KEY (id_categorie),
   CONSTRAINT uq_categorii_nume UNIQUE (nume),
   CONSTRAINT uq_categorii_cod UNIQUE (cod)
```

```
);


CREATE TABLE PRODUSE (
   id_produs          INT,
   id_categorie       INT NOT NULL, -- un produs trebuie sa apartina obligatoriu unei categorii
   denumire           VARCHAR2(100) NOT NULL,
   pret               NUMBER(10, 2) NOT NULL,
   stoc               INT DEFAULT 0 NOT NULL,
   CONSTRAINT pk_produse PRIMARY KEY (id_produs),
   CONSTRAINT fk_produse_categorii FOREIGN KEY (id_categorie) REFERENCES
CATEGORII(id_categorie),
   CONSTRAINT ck_produse_pret CHECK (pret > 0),
   CONSTRAINT ck_produse_stoc CHECK (stoc >= 0)
);


CREATE TABLE FURNIZORI (
   id_furnizor        INT,
   denumire           VARCHAR2(100) NOT NULL,
   email              VARCHAR2(100) NOT NULL,
   telefon            VARCHAR2(15),
   data_contract      DATE DEFAULT SYSDATE NOT NULL,
   CONSTRAINT pk_furnizori PRIMARY KEY (id_furnizor),
   CONSTRAINT uq_furnizori_email UNIQUE (email),
   CONSTRAINT ck_furnizori_email CHECK (email LIKE '%@%.%')
);
/


CREATE OR REPLACE TRIGGER trg_check_data_contract
BEFORE INSERT OR UPDATE ON FURNIZORI
FOR EACH ROW
BEGIN
   IF :NEW.data_contract > SYSDATE THEN
      RAISE_APPLICATION_ERROR(-20002, 'Data contractului nu poate fi in viitor!');
   END IF;
END;
/


CREATE TABLE COMENZI (
   id_comanda         INT,
   id_client          INT NOT NULL, -- o comanda trebuie sa apartina obligatoriu unui client
   data_comanda       DATE DEFAULT SYSDATE NOT NULL,
   status             VARCHAR2(20) DEFAULT 'in asteptare' NOT NULL,
   metoda_plata       VARCHAR2(20),
```

```
   CONSTRAINT pk_comenzi PRIMARY KEY (id_comanda),
   CONSTRAINT fk_comenzi_clienti FOREIGN KEY (id_client) REFERENCES CLIENTI(id_client),
   CONSTRAINT ck_comenzi_status CHECK (status IN ('in asteptare', 'livrata', 'anulata')),
   CONSTRAINT ck_comenzi_plata CHECK (metoda_plata IN ('card', 'numerar'))
);
/


CREATE OR REPLACE TRIGGER trg_check_data_comanda
BEFORE INSERT OR UPDATE ON COMENZI
FOR EACH ROW
DECLARE
   v_data_inreg DATE;
BEGIN
   -- Validare fata de timpul prezent
   IF :NEW.data_comanda > SYSDATE THEN
      RAISE_APPLICATION_ERROR(-20003, 'Eroare: Data comenzii nu poate fi in viitor!');
   END IF;

   -- Validare fata de inregistrarea clientului
   SELECT data_inregistrare INTO v_data_inreg
   FROM CLIENTI
   WHERE id_client = :NEW.id_client;

   IF :NEW.data_comanda < v_data_inreg THEN
      RAISE_APPLICATION_ERROR(-20101, 'Eroare: Clientul nu poate plasa comenzi inainte de data
inregistrarii sale (' || TO_CHAR(v_data_inreg, 'DD-MM-YYYY') || ')!');
   END IF;
END;
/

CREATE OR REPLACE TRIGGER trg_status_comanda_stoc
AFTER UPDATE OF status ON COMENZI
FOR EACH ROW
BEGIN
   -- Comanda este anulata (returnam produsele in stoc)
   IF :NEW.status = 'anulata' AND :OLD.status != 'anulata' THEN
      FOR r_item IN (SELECT id_produs, cantitate FROM DETALII_COMANDA WHERE id_comanda
= :NEW.id_comanda) LOOP
         UPDATE PRODUSE
         SET stoc = stoc + r_item.cantitate
         WHERE id_produs = r_item.id_produs;
      END LOOP;
      DBMS_OUTPUT.PUT_LINE('Comanda ' || :NEW.id_comanda || ' a fost anulata. Produsele au
revenit in stoc.');

   -- Comanda revine din anulare (scadem din nou produsele din stoc)
```

```
   ELSIF :OLD.status = 'anulata' AND (:NEW.status = 'in asteptare' OR :NEW.status = 'livrata') THEN
      FOR r_item IN (SELECT id_produs, cantitate FROM DETALII_COMANDA WHERE id_comanda
= :NEW.id_comanda) LOOP
         UPDATE PRODUSE
         SET stoc = stoc - r_item.cantitate
         WHERE id_produs = r_item.id_produs;
      END LOOP;
      DBMS_OUTPUT.PUT_LINE('Comanda ' || :NEW.id_comanda || ' a fost reactivata. Stocul a fost
actualizat.');
   END IF;
END;
/


CREATE TABLE FACTURI (
   id_factura        INT,
   id_comanda        INT NOT NULL, -- o factura trebuie sa fie emisa pentru o comanda
   data_emitere      DATE DEFAULT SYSDATE NOT NULL,
   status_plata      VARCHAR2(20) DEFAULT 'neplatita' NOT NULL,
   CONSTRAINT pk_facturi PRIMARY KEY (id_factura),
   CONSTRAINT fk_facturi_comenzi FOREIGN KEY (id_comanda) REFERENCES
COMENZI(id_comanda),
   CONSTRAINT ck_facturi_status CHECK (status_plata IN ('platita', 'neplatita'))
);
/


CREATE OR REPLACE TRIGGER trg_check_data_factura
BEFORE INSERT OR UPDATE ON FACTURI
FOR EACH ROW
DECLARE
   v_data_comanda DATE;
BEGIN
   -- Validare fata de timpul prezent
   IF :NEW.data_emitere > SYSDATE THEN
      RAISE_APPLICATION_ERROR(-20004, 'Eroare: Data emiterii facturii nu poate fi in viitor!');
   END IF;

   -- Validare fata de data comenzii
   SELECT data_comanda INTO v_data_comanda
   FROM COMENZI
   WHERE id_comanda = :NEW.id_comanda;

   IF :NEW.data_emitere < v_data_comanda THEN
      RAISE_APPLICATION_ERROR(-20102, 'Eroare: Factura nu poate fi emisa inainte de plasarea
comenzii (' || TO_CHAR(v_data_comanda, 'DD-MM-YYYY') || ')!');
   END IF;
```

```
END;
/


CREATE TABLE DETALII_COMANDA ( -- retine ce produse contine o comanda si in ce cantitate
   id_comanda        INT,
   id_produs         INT,
   cantitate         INT NOT NULL,
   pret_unitar       NUMBER(10, 2) NOT NULL, -- pretul produsului la momentul comenzii
   CONSTRAINT pk_detalii_comanda PRIMARY KEY (id_comanda, id_produs),
   CONSTRAINT fk_detalii_comanda_comenzi FOREIGN KEY (id_comanda) REFERENCES
COMENZI(id_comanda),
   CONSTRAINT fk_detalii_comanda_produse FOREIGN KEY (id_produs) REFERENCES
PRODUSE(id_produs),
   CONSTRAINT ck_detalii_cantitate CHECK (cantitate >= 1),
   CONSTRAINT ck_detalii_pret CHECK (pret_unitar > 0)
);
/

CREATE OR REPLACE TRIGGER trg_blocare_detalii_livrate
BEFORE INSERT OR UPDATE OR DELETE ON DETALII_COMANDA
FOR EACH ROW
DECLARE
   v_status VARCHAR2(20);
   v_id_com_temp INT;
BEGIN
   -- Aflam ID-ul comenzii pe care se incearca operatia
   IF INSERTING OR UPDATING THEN
      v_id_com_temp := :NEW.id_comanda;
   ELSE
      v_id_com_temp := :OLD.id_comanda;
   END IF;

   -- Identificam statusul comenzii asociate din tabelul COMENZI
   SELECT status INTO v_status
   FROM COMENZI
   WHERE id_comanda = v_id_com_temp;

   -- Daca statusul este livrat, blocam orice modificare
   IF v_status = 'livrata' THEN
      RAISE_APPLICATION_ERROR(-20011, 'Eroare: Nu se pot adauga, modifica sau sterge produse
dintr-o comanda care a fost deja livrata!');
   END IF;

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RAISE_APPLICATION_ERROR(-20012, 'Eroare: Comanda asociata ID-ului ' || v_id_com_temp || '
```

```
nu a fost gasita!');
END;
/

-- Trigger pentru actualizarea stocului -- cerinta 11, declansat de INSERT-urile / UPDATE-urile /
DELETE-urile din DETALII_COMANDA
CREATE OR REPLACE TRIGGER trg_gestiune_stoc_vanzari
BEFORE INSERT OR UPDATE OR DELETE ON DETALII_COMANDA
FOR EACH ROW
FOLLOWS trg_blocare_detalii_livrate
DECLARE
   v_stoc_curent INT;
   v_id_produs   INT;
BEGIN
   -- Identificam ID-ul produsului (cel nou la insert/update, cel vechi la delete)
   v_id_produs := CASE WHEN DELETING THEN :OLD.id_produs ELSE :NEW.id_produs END;

   -- Selectam stocul curent cu blocare (FOR UPDATE - blocam randul in situatia in care doi clienti
comanda ultimul produs in acelasi timp)
   BEGIN
      SELECT stoc INTO v_stoc_curent
      FROM PRODUSE
      WHERE id_produs = v_id_produs
      FOR UPDATE;
   EXCEPTION
      WHEN NO_DATA_FOUND THEN
         RAISE_APPLICATION_ERROR(-20006, 'Produsul cu ID ' || v_id_produs || ' nu a fost gasit!');
   END;

   IF INSERTING THEN
      -- Verificam daca avem destul stoc pentru noua vanzare
      IF v_stoc_curent < :NEW.cantitate THEN
         RAISE_APPLICATION_ERROR(-20005, 'Stoc insuficient! Disponibil: ' || v_stoc_curent);
      END IF;

      UPDATE PRODUSE SET stoc = stoc - :NEW.cantitate WHERE id_produs = v_id_produs;

   ELSIF UPDATING THEN
      -- Verificam daca noul stoc (stoc_curent + ce era deja rezervat - ce se cere acum) este suficient
      -- Ex. - Avem 10 in stoc, am comandat 5 (deci in total erau 15). Modificam la 12
      -- Verificam daca 15 >= 12
      IF (v_stoc_curent + :OLD.cantitate) < :NEW.cantitate THEN
          RAISE_APPLICATION_ERROR(-20005, 'Stoc insuficient pentru modificare! Maxim disponibil: '
|| (v_stoc_curent + :OLD.cantitate));
      END IF;

      UPDATE PRODUSE SET stoc = stoc + (:OLD.cantitate - :NEW.cantitate) WHERE id_produs =
```

```
v_id_produs;

   ELSIF DELETING THEN
      -- Punem produsele inapoi in stoc daca se sterge comanda
      UPDATE PRODUSE SET stoc = stoc + :OLD.cantitate WHERE id_produs = v_id_produs;
   END IF;
END;
/


CREATE TABLE PRODUSE_FURNIZORI ( -- ce furnizor a livrat ce produs, cand si in ce cantitate
   id_produs          INT,
   id_furnizor        INT,
   data_inceput       DATE DEFAULT SYSDATE NOT NULL, -- marcheaza data achizitiei/inceput
contract
   data_sfarsit       DATE,
   stoc_achizitie     INT NOT NULL, -- cantitatea de produs aprovizionata de la furnizor
   pret_achizitie     NUMBER(10, 2) NOT NULL, -- pretul de achizitie al produsului de la furnizor
   CONSTRAINT pk_produse_furnizori PRIMARY KEY (id_produs, id_furnizor, data_inceput),
   CONSTRAINT fk_pf_produse FOREIGN KEY (id_produs) REFERENCES PRODUSE(id_produs),
   CONSTRAINT fk_pf_furnizori FOREIGN KEY (id_furnizor) REFERENCES
FURNIZORI(id_furnizor),
   CONSTRAINT ck_pf_stoc CHECK (stoc_achizitie >= 1),
   CONSTRAINT ck_pf_pret CHECK (pret_achizitie >= 0),
   CONSTRAINT ck_pf_perioada CHECK (data_sfarsit IS NULL OR data_sfarsit >= data_inceput)
);
/


CREATE OR REPLACE TRIGGER trg_check_dates_pf
BEFORE INSERT OR UPDATE ON PRODUSE_FURNIZORI
FOR EACH ROW
DECLARE
   v_data_contract DATE;
BEGIN
   -- Validare fata de timpul prezent
   IF :NEW.data_inceput > SYSDATE THEN
      RAISE_APPLICATION_ERROR(-20007, 'Eroare: Data de inceput a aprovizionarii nu poate fi in
viitor!');
   END IF;

   -- Validare fata de data contractului
   SELECT data_contract INTO v_data_contract
   FROM FURNIZORI
   WHERE id_furnizor = :NEW.id_furnizor;

   IF :NEW.data_inceput < v_data_contract THEN
```

```
        RAISE_APPLICATION_ERROR(-20105, 'Eroare: Nu se pot receptiona produse inainte de semnarea
contractului cu furnizorul!');
    END IF;

    -- Validare consistenta perioada (daca exista data_sfarsit)
    IF :NEW.data_sfarsit IS NOT NULL THEN
        IF :NEW.data_sfarsit > SYSDATE THEN
            RAISE_APPLICATION_ERROR(-20008, 'Eroare: Data de sfarsit nu poate fi in viitor!');
        ELSIF :NEW.data_sfarsit < :NEW.data_inceput THEN
            RAISE_APPLICATION_ERROR(-20106, 'Eroare: Data de sfarsit trebuie sa fie dupa data de
inceput!');
        END IF;
    END IF;
END;
/

-- Trigger pentru actualizarea stocului (aprovizionare) - cerinta 11, declansat de INSERT-urile /
UPDATE-urile / DELETE-urile din PRODUSE_FURNIZORI
CREATE OR REPLACE TRIGGER trg_gestiune_stoc_furnizor
AFTER INSERT OR UPDATE OR DELETE ON PRODUSE_FURNIZORI
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        -- Adaugam intreaga cantitate noua
        UPDATE PRODUSE
        SET stoc = stoc + :NEW.stoc_achizitie
        WHERE id_produs = :NEW.id_produs;

    ELSIF UPDATING THEN
        -- Adaugam diferenta (Nou - Vechi)
        -- Daca Nou > Vechi, stocul creste. Daca Nou < Vechi, stocul scade automat.
        UPDATE PRODUSE
        SET stoc = stoc + (:NEW.stoc_achizitie - :OLD.stoc_achizitie)
        WHERE id_produs = :NEW.id_produs;

    ELSIF DELETING THEN
        -- Daca stergem o achizitie, scadem cantitatea din stoc
        UPDATE PRODUSE
        SET stoc = stoc - :OLD.stoc_achizitie
        WHERE id_produs = :OLD.id_produs;
    END IF;
END;
/



CREATE TABLE PROCESARE_COMENZI ( -- pastreaza istoricul procesarii
```

```
   id_angajat         INT,
   id_comanda         INT,
   data_proc          DATE DEFAULT SYSDATE NOT NULL,
   CONSTRAINT pk_procesare_comenzi PRIMARY KEY (id_angajat, id_comanda),
   CONSTRAINT fk_proc_angajati FOREIGN KEY (id_angajat) REFERENCES
ANGAJATI(id_angajat),
   CONSTRAINT fk_proc_comenzi FOREIGN KEY (id_comanda) REFERENCES
COMENZI(id_comanda)
);
/

CREATE OR REPLACE TRIGGER trg_check_data_proc
BEFORE INSERT OR UPDATE ON PROCESARE_COMENZI
FOR EACH ROW
DECLARE
   v_data_angajare DATE;
   v_data_comanda   DATE;
BEGIN
   -- Validare fata de timpul prezent
   IF :NEW.data_proc > SYSDATE THEN
      RAISE_APPLICATION_ERROR(-20009, 'Eroare: Data procesarii nu poate fi in viitor!');
   END IF;

   -- Validare fata de data angajarii
   SELECT data_angajare INTO v_data_angajare
   FROM ANGAJATI
   WHERE id_angajat = :NEW.id_angajat;

   IF :NEW.data_proc < v_data_angajare THEN
      RAISE_APPLICATION_ERROR(-20103, 'Eroare: Angajatul nu putea procesa comenzi inainte de a fi
angajat!');
   END IF;

   -- Validare fata de data comenzii
   SELECT data_comanda INTO v_data_comanda
   FROM COMENZI
   WHERE id_comanda = :NEW.id_comanda;

   IF :NEW.data_proc < v_data_comanda THEN
      RAISE_APPLICATION_ERROR(-20104, 'Eroare: Procesarea nu poate avea loc inainte de plasarea
comenzii!');
   END IF;
END;
/
```

```oracle_plsql
-- Cerinta 4

CREATE TABLE CLIENTI (
    id_client           INT,
    nume                VARCHAR2(50) NOT NULL,
    prenume             VARCHAR2(50) NOT NULL,
    email               VARCHAR2(100) NOT NULL,
    telefon             VARCHAR2(15),
    data_inregistrare   DATE DEFAULT SYSDATE NOT NULL,
    CONSTRAINT pk_clienti PRIMARY KEY (id_client),
    CONSTRAINT uq_clienti_email UNIQUE (email),
    CONSTRAINT ck_clienti_email CHECK (email LIKE '%@%.%')
);


CREATE TABLE ANGAJATI (
    id_angajat          INT,
    nume                VARCHAR2(50) NOT NULL,
    prenume             VARCHAR2(50) NOT NULL,
    email               VARCHAR2(100) NOT NULL,
    telefon             VARCHAR2(15),
    data_angajare       DATE DEFAULT SYSDATE NOT NULL,
    CONSTRAINT pk_angajati PRIMARY KEY (id_angajat),
    CONSTRAINT uq_angajati_email UNIQUE (email),
    CONSTRAINT ck_angajati_email CHECK (email LIKE '%@%.%')
);

CREATE OR REPLACE TRIGGER trg_check_data_angajare -- cerinta 11, declansat de INSERT-urile din ANGAJATI
BEFORE INSERT OR UPDATE ON ANGAJATI
FOR EACH ROW
BEGIN
    IF :NEW.data_angajare > SYSDATE THEN
        RAISE_APPLICATION_ERROR(-20001, 'Data angajarii nu poate fi in viitor!');
    END IF;
END;
/
```

```sql
CREATE TABLE CATEGORII (
    id_categorie        INT,
    nume                VARCHAR2(50) NOT NULL,
    cod                 VARCHAR2(10) NOT NULL,
    CONSTRAINT pk_categorii PRIMARY KEY (id_categorie),
    CONSTRAINT uq_categorii_nume UNIQUE (nume),
    CONSTRAINT uq_categorii_cod UNIQUE (cod)
);


CREATE TABLE PRODUSE (
    id_produs           INT,
    id_categorie        INT NOT NULL, -- un produs trebuie sa apartina obligatoriu unei categorii
    denumire            VARCHAR2(100) NOT NULL,
    pret                NUMBER(10, 2) NOT NULL,
    stoc                INT DEFAULT 0 NOT NULL,
    CONSTRAINT pk_produse PRIMARY KEY (id_produs),
    CONSTRAINT fk_produse_categorii FOREIGN KEY (id_categorie) REFERENCES CATEGORII(id_categorie),
    CONSTRAINT ck_produse_pret CHECK (pret > 0),
    CONSTRAINT ck_produse_stoc CHECK (stoc >= 0)
);


CREATE TABLE FURNIZORI (
    id_furnizor         INT,
    denumire            VARCHAR2(100) NOT NULL,
    email               VARCHAR2(100) NOT NULL,
    telefon             VARCHAR2(15),
    data_contract       DATE DEFAULT SYSDATE NOT NULL,
    CONSTRAINT pk_furnizori PRIMARY KEY (id_furnizor),
    CONSTRAINT uq_furnizori_email UNIQUE (email),
    CONSTRAINT ck_furnizori_email CHECK (email LIKE '%@%.%')
);
/
```

```oracle_plsql
CREATE OR REPLACE TRIGGER trg_check_data_contract
BEFORE INSERT OR UPDATE ON FURNIZORI
FOR EACH ROW
BEGIN
    IF :NEW.data_contract > SYSDATE THEN
        RAISE_APPLICATION_ERROR(-20002, 'Data contractului nu poate fi in viitor!');
    END IF;
END;
/


CREATE TABLE COMENZI (
    id_comanda          INT,
    id_client           INT NOT NULL, -- o comanda trebuie sa apartina obligatoriu unui client
    data_comanda        DATE DEFAULT SYSDATE NOT NULL,
    status              VARCHAR2(20) DEFAULT 'in asteptare' NOT NULL,
    metoda_plata        VARCHAR2(20),
    CONSTRAINT pk_comenzi PRIMARY KEY (id_comanda),
    CONSTRAINT fk_comenzi_clienti FOREIGN KEY (id_client) REFERENCES CLIENTI(id_client),
    CONSTRAINT ck_comenzi_status CHECK (status IN ('in asteptare', 'livrata', 'anulata')),
    CONSTRAINT ck_comenzi_plata CHECK (metoda_plata IN ('card', 'numerar'))
);
/


CREATE OR REPLACE TRIGGER trg_check_data_comanda
BEFORE INSERT OR UPDATE ON COMENZI
FOR EACH ROW
DECLARE
    v_data_inreg DATE;
BEGIN
    -- Validare fata de timpul prezent
    IF :NEW.data_comanda > SYSDATE THEN
        RAISE_APPLICATION_ERROR(-20003, 'Eroare: Data comenzii nu poate fi in viitor!');
    END IF;
```

```oracle_plsql
    -- Validare fata de inregistrarea clientului
    SELECT data_inregistrare INTO v_data_inreg
    FROM CLIENTI
    WHERE id_client = :NEW.id_client;

    IF :NEW.data_comanda < v_data_inreg THEN
        RAISE_APPLICATION_ERROR(-20101, 'Eroare: Clientul nu poate plasa comenzi inainte de data inregistrarii sale (' || TO_CHAR(v_data_inreg, 'DD-MM-YYYY') || ')!');
    END IF;
END;
/

CREATE OR REPLACE TRIGGER trg_status_comanda_stoc
AFTER UPDATE OF status ON COMENZI
FOR EACH ROW
BEGIN
    -- Comanda este anulata (returnam produsele in stoc)
    IF :NEW.status = 'anulata' AND :OLD.status != 'anulata' THEN
        FOR r_item IN (SELECT id_produs, cantitate FROM DETALII_COMANDA WHERE id_comanda = :NEW.id_comanda) LOOP
            UPDATE PRODUSE
            SET stoc = stoc + r_item.cantitate
            WHERE id_produs = r_item.id_produs;
        END LOOP;
        DBMS_OUTPUT.PUT_LINE('Comanda ' || :NEW.id_comanda || ' a fost anulata. Produsele au revenit in stoc.');

    -- Comanda revine din anulare (scadem din nou produsele din stoc)
    ELSIF :OLD.status = 'anulata' AND (:NEW.status = 'in asteptare' OR :NEW.status = 'livrata') THEN
        FOR r_item IN (SELECT id_produs, cantitate FROM DETALII_COMANDA WHERE id_comanda = :NEW.id_comanda) LOOP
            UPDATE PRODUSE
            SET stoc = stoc - r_item.cantitate
            WHERE id_produs = r_item.id_produs;
        END LOOP;
        DBMS_OUTPUT.PUT_LINE('Comanda ' || :NEW.id_comanda || ' a fost reactivata. Stocul a fost actualizat.');
    END IF;
END;
/
```

```oracle_plsql


CREATE TABLE FACTURI (
    id_factura          INT,
    id_comanda          INT NOT NULL, -- o factura trebuie sa fie emisa pentru o comanda
    data_emitere        DATE DEFAULT SYSDATE NOT NULL,
    status_plata        VARCHAR2(20) DEFAULT 'neplatita' NOT NULL,
    CONSTRAINT pk_facturi PRIMARY KEY (id_factura),
    CONSTRAINT fk_facturi_comenzi FOREIGN KEY (id_comanda) REFERENCES COMENZI(id_comanda),
    CONSTRAINT ck_facturi_status CHECK (status_plata IN ('platita', 'neplatita'))
);
/


CREATE OR REPLACE TRIGGER trg_check_data_factura
BEFORE INSERT OR UPDATE ON FACTURI
FOR EACH ROW
DECLARE
    v_data_comanda DATE;
BEGIN
    -- Validare fata de timpul prezent
    IF :NEW.data_emitere > SYSDATE THEN
        RAISE_APPLICATION_ERROR(-20004, 'Eroare: Data emiterii facturii nu poate fi in viitor!');
    END IF;

    -- Validare fata de data comenzii
    SELECT data_comanda INTO v_data_comanda
    FROM COMENZI
    WHERE id_comanda = :NEW.id_comanda;

    IF :NEW.data_emitere < v_data_comanda THEN
        RAISE_APPLICATION_ERROR(-20102, 'Eroare: Factura nu poate fi emisa inainte de plasarea comenzii (' || TO_CHAR(v_data_comanda, 'DD-MM-YYYY') || ')!');
    END IF;
END;
/

```

```oracle_plsql

CREATE TABLE DETALII_COMANDA ( -- retine ce produse contine o comanda si in ce cantitate
    id_comanda          INT,
    id_produs           INT,
    cantitate           INT NOT NULL,
    pret_unitar         NUMBER(10, 2) NOT NULL, -- pretul produsului la momentul comenzii
    CONSTRAINT pk_detalii_comanda PRIMARY KEY (id_comanda, id_produs),
    CONSTRAINT fk_detalii_comanda_comenzi FOREIGN KEY (id_comanda) REFERENCES COMENZI(id_comanda),
    CONSTRAINT fk_detalii_comanda_produse FOREIGN KEY (id_produs) REFERENCES PRODUSE(id_produs),
    CONSTRAINT ck_detalii_cantitate CHECK (cantitate >= 1),
    CONSTRAINT ck_detalii_pret CHECK (pret_unitar > 0)
);
/

CREATE OR REPLACE TRIGGER trg_blocare_detalii_livrate
BEFORE INSERT OR UPDATE OR DELETE ON DETALII_COMANDA
FOR EACH ROW
DECLARE
    v_status VARCHAR2(20);
    v_id_com_temp INT;
BEGIN
    -- Aflam ID-ul comenzii pe care se incearca operatia
    IF INSERTING OR UPDATING THEN
        v_id_com_temp := :NEW.id_comanda;
    ELSE
        v_id_com_temp := :OLD.id_comanda;
    END IF;

    -- Identificam statusul comenzii asociate din tabelul COMENZI
    SELECT status INTO v_status
    FROM COMENZI
    WHERE id_comanda = v_id_com_temp;

    -- Daca statusul este livrat, blocam orice modificare
    IF v_status = 'livrata' THEN
        RAISE_APPLICATION_ERROR(-20011, 'Eroare: Nu se pot adauga, modifica sau sterge produse dintr-o comanda care a fost deja livrata!');
```

```oracle_plsql
    END IF;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RAISE_APPLICATION_ERROR(-20012, 'Eroare: Comanda asociata ID-ului ' || v_id_com_temp || ' nu a fost gasita!');
END;
/

-- Trigger pentru actualizarea stocului -- cerinta 11, declansat de INSERT-urile / UPDATE-urile / DELETE-urile din DETALII_COMANDA
CREATE OR REPLACE TRIGGER trg_gestiune_stoc_vanzari
BEFORE INSERT OR UPDATE OR DELETE ON DETALII_COMANDA
FOR EACH ROW
FOLLOWS trg_blocare_detalii_livrate
DECLARE
    v_stoc_curent INT;
    v_id_produs   INT;
BEGIN
    -- Identificam ID-ul produsului (cel nou la insert/update, cel vechi la delete)
    v_id_produs := CASE WHEN DELETING THEN :OLD.id_produs ELSE :NEW.id_produs END;

    -- Selectam stocul curent cu blocare (FOR UPDATE - blocam randul in situatia in care doi clienti comanda ultimul produs in acelasi timp)
    BEGIN
        SELECT stoc INTO v_stoc_curent
        FROM PRODUSE
        WHERE id_produs = v_id_produs
        FOR UPDATE;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20006, 'Produsul cu ID ' || v_id_produs || ' nu a fost gasit!');
    END;

    IF INSERTING THEN
        -- Verificam daca avem destul stoc pentru noua vanzare
        IF v_stoc_curent < :NEW.cantitate THEN
            RAISE_APPLICATION_ERROR(-20005, 'Stoc insuficient! Disponibil: ' || v_stoc_curent);
        END IF;

        UPDATE PRODUSE SET stoc = stoc - :NEW.cantitate WHERE id_produs = v_id_produs;

    ELSIF UPDATING THEN
        -- Verificam daca noul stoc (stoc_curent + ce era deja rezervat - ce se cere acum) este suficient
        -- Ex. - Avem 10 in stoc, am comandat 5 (deci in total erau 15). Modificam la 12
        -- Verificam daca 15 >= 12
        IF (v_stoc_curent + :OLD.cantitate) < :NEW.cantitate THEN
            RAISE_APPLICATION_ERROR(-20005, 'Stoc insuficient pentru modificare! Maxim disponibil: ' || (v_stoc_curent + :OLD.cantitate));
        END IF;

        UPDATE PRODUSE SET stoc = stoc + (:OLD.cantitate - :NEW.cantitate) WHERE id_produs = v_id_produs;

    ELSIF DELETING THEN
        -- Punem produsele inapoi in stoc daca se sterge comanda
        UPDATE PRODUSE SET stoc = stoc + :OLD.cantitate WHERE id_produs = v_id_produs;
    END IF;
END;
/


CREATE TABLE PRODUSE_FURNIZORI ( -- ce furnizor a livrat ce produs, cand si in ce cantitate
    id_produs           INT,
    id_furnizor         INT,
    data_inceput        DATE DEFAULT SYSDATE NOT NULL, -- marcheaza data achizitiei/inceput contract
    data_sfarsit        DATE,
    stoc_achizitie      INT NOT NULL, -- cantitatea de produs aprovizionata de la furnizor
    pret_achizitie      NUMBER(10, 2) NOT NULL, -- pretul de achizitie al produsului de la furnizor
    CONSTRAINT pk_produse_furnizori PRIMARY KEY (id_produs, id_furnizor, data_inceput),
    CONSTRAINT fk_pf_produse FOREIGN KEY (id_produs) REFERENCES PRODUSE(id_produs),
    CONSTRAINT fk_pf_furnizori FOREIGN KEY (id_furnizor) REFERENCES FURNIZORI(id_furnizor),
    CONSTRAINT ck_pf_stoc CHECK (stoc_achizitie >= 1),
```

```oracle_plsql
    CONSTRAINT ck_pf_pret CHECK (pret_achizitie >= 0),
    CONSTRAINT ck_pf_perioada CHECK (data_sfarsit IS NULL OR data_sfarsit >= data_inceput)
);
/


CREATE OR REPLACE TRIGGER trg_check_dates_pf
BEFORE INSERT OR UPDATE ON PRODUSE_FURNIZORI
FOR EACH ROW
DECLARE
    v_data_contract DATE;
BEGIN
    -- Validare fata de timpul prezent
    IF :NEW.data_inceput > SYSDATE THEN
        RAISE_APPLICATION_ERROR(-20007, 'Eroare: Data de inceput a aprovizionarii nu poate fi in viitor!');
    END IF;

    -- Validare fata de data contractului
    SELECT data_contract INTO v_data_contract
    FROM FURNIZORI
    WHERE id_furnizor = :NEW.id_furnizor;

    IF :NEW.data_inceput < v_data_contract THEN
        RAISE_APPLICATION_ERROR(-20105, 'Eroare: Nu se pot receptiona produse inainte de semnarea contractului cu furnizorul!');
    END IF;

    -- Validare consistenta perioada (daca exista data_sfarsit)
    IF :NEW.data_sfarsit IS NOT NULL THEN
        IF :NEW.data_sfarsit > SYSDATE THEN
            RAISE_APPLICATION_ERROR(-20008, 'Eroare: Data de sfarsit nu poate fi in viitor!');
        ELSIF :NEW.data_sfarsit < :NEW.data_inceput THEN
            RAISE_APPLICATION_ERROR(-20106, 'Eroare: Data de sfarsit trebuie sa fie dupa data de inceput!');
        END IF;
```

```oracle_plsql
    END IF;
END;
/

-- Trigger pentru actualizarea stocului (aprovizionare) - cerinta 11, declansat de INSERT-urile / UPDATE-urile / DELETE-urile din PRODUSE_FURNIZORI
CREATE OR REPLACE TRIGGER trg_gestiune_stoc_furnizor
AFTER INSERT OR UPDATE OR DELETE ON PRODUSE_FURNIZORI
FOR EACH ROW
BEGIN
    IF INSERTING THEN
        -- Adaugam intreaga cantitate noua
        UPDATE PRODUSE
        SET stoc = stoc + :NEW.stoc_achizitie
        WHERE id_produs = :NEW.id_produs;

    ELSIF UPDATING THEN
        -- Adaugam diferenta (Nou - Vechi)
        -- Daca Nou > Vechi, stocul creste. Daca Nou < Vechi, stocul scade automat.
        UPDATE PRODUSE
        SET stoc = stoc + (:NEW.stoc_achizitie - :OLD.stoc_achizitie)
        WHERE id_produs = :NEW.id_produs;

    ELSIF DELETING THEN
        -- Daca stergem o achizitie, scadem cantitatea din stoc
        UPDATE PRODUSE
        SET stoc = stoc - :OLD.stoc_achizitie
        WHERE id_produs = :OLD.id_produs;
    END IF;
END;
/
```

```oracle_plsql
CREATE TABLE PROCESARE_COMENZI ( -- pastreaza istoricul procesarii
    id_angajat          INT,
    id_comanda          INT,
    data_proc           DATE DEFAULT SYSDATE NOT NULL,
    CONSTRAINT pk_procesare_comenzi PRIMARY KEY (id_angajat, id_comanda),
    CONSTRAINT fk_proc_angajati FOREIGN KEY (id_angajat) REFERENCES ANGAJATI(id_angajat),
    CONSTRAINT fk_proc_comenzi FOREIGN KEY (id_comanda) REFERENCES COMENZI(id_comanda)
);
/

CREATE OR REPLACE TRIGGER trg_check_data_proc
BEFORE INSERT OR UPDATE ON PROCESARE_COMENZI
FOR EACH ROW
DECLARE
    v_data_angajare DATE;
    v_data_comanda   DATE;
BEGIN
    -- Validare fata de timpul prezent
    IF :NEW.data_proc > SYSDATE THEN
        RAISE_APPLICATION_ERROR(-20009, 'Eroare: Data procesarii nu poate fi in viitor!');
    END IF;

    -- Validare fata de data angajarii
    SELECT data_angajare INTO v_data_angajare
    FROM ANGAJATI
    WHERE id_angajat = :NEW.id_angajat;

    IF :NEW.data_proc < v_data_angajare THEN
        RAISE_APPLICATION_ERROR(-20103, 'Eroare: Angajatul nu putea procesa comenzi inainte de a fi angajat!');
    END IF;

    -- Validare fata de data comenzii
    SELECT data_comanda INTO v_data_comanda
    FROM COMENZI
    WHERE id_comanda = :NEW.id_comanda;

```

```oracle_plsql
    IF :NEW.data_proc < v_data_comanda THEN
        RAISE_APPLICATION_ERROR(-20104, 'Eroare: Procesarea nu poate avea loc inainte de plasarea comenzii!');
    END IF;
END;
/

```

5. **Adăugarea de informații coerente în tabelele create (minimum 5 înregistrări pentru fiecare entitate independentă; minimum 10 înregistrări pentru fiecare tabel asociativ)**

```
-- SECVENTE
-- Stergem secventele vechi daca exista
BEGIN
  EXECUTE IMMEDIATE 'DROP SEQUENCE seq_categorii';
  EXECUTE IMMEDIATE 'DROP SEQUENCE seq_produse';
  EXECUTE IMMEDIATE 'DROP SEQUENCE seq_furnizori';
  EXECUTE IMMEDIATE 'DROP SEQUENCE seq_clienti';
  EXECUTE IMMEDIATE 'DROP SEQUENCE seq_angajati';
  EXECUTE IMMEDIATE 'DROP SEQUENCE seq_comenzi';
  EXECUTE IMMEDIATE 'DROP SEQUENCE seq_facturi';
EXCEPTION
  WHEN OTHERS THEN NULL; -- Ignora eroarea daca secventele nu exista
END;
/

-- Creare Secvente
CREATE SEQUENCE seq_categorii START WITH 1 INCREMENT BY 1;
CREATE SEQUENCE seq_produse   START WITH 101 INCREMENT BY 1;
CREATE SEQUENCE seq_furnizori START WITH 10 INCREMENT BY 10; -- Furnizorii vor fi 10, 20,
30...
CREATE SEQUENCE seq_clienti   START WITH 1001 INCREMENT BY 1;
CREATE SEQUENCE seq_angajati  START WITH 501 INCREMENT BY 1;
CREATE SEQUENCE seq_comenzi   START WITH 2001 INCREMENT BY 1;
CREATE SEQUENCE seq_facturi   START WITH 3001 INCREMENT BY 1;


-- Tabele independente


-- CATEGORII (ID-urile generate vor fi 1, 2, 3, 4, 5)
INSERT INTO CATEGORII (id_categorie, nume, cod) VALUES (seq_categorii.NEXTVAL, 'Telefoane
Mobile', 'TEL');
INSERT INTO CATEGORII (id_categorie, nume, cod) VALUES (seq_categorii.NEXTVAL, 'Laptopuri
& IT', 'LAP');
INSERT INTO CATEGORII (id_categorie, nume, cod) VALUES (seq_categorii.NEXTVAL,
'Televizoare & Audio', 'TV');
INSERT INTO CATEGORII (id_categorie, nume, cod) VALUES (seq_categorii.NEXTVAL,
'Electrocasnice Mici', 'ELECTRO');
INSERT INTO CATEGORII (id_categorie, nume, cod) VALUES (seq_categorii.NEXTVAL, 'Gaming',
'GAME');

-- PRODUSE (ID-urile generate vor fi 101, 102 ... 110)
-- la id_categorie folosim valorile 1-5 generate mai sus.
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES
```

```
(seq_produse.NEXTVAL, 1, 'Samsung Galaxy S23', 3999.99, 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES
(seq_produse.NEXTVAL, 1, 'iPhone 15 Pro', 5400.00, 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES
(seq_produse.NEXTVAL, 2, 'Laptop ASUS TUF Gaming', 4500.50, 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES
(seq_produse.NEXTVAL, 2, 'MacBook Air M2', 5999.00, 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES
(seq_produse.NEXTVAL, 3, 'Televizor LG OLED', 6200.00, 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES
(seq_produse.NEXTVAL, 4, 'Espresor Philips LatteGo', 2100.00, 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES
(seq_produse.NEXTVAL, 4, 'Aspirator Robot Roborock', 1850.00, 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES
(seq_produse.NEXTVAL, 5, 'Consola PlayStation 5', 2699.99, 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES
(seq_produse.NEXTVAL, 2, 'Mouse Logitech MX Master', 450.00, 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES
(seq_produse.NEXTVAL, 3, 'Soundbar Samsung', 899.00, 0);

-- FURNIZORI (ID-urile generate vor fi 10, 20, 30, 40, 50)
INSERT INTO FURNIZORI (id_furnizor, denumire, email, telefon, data_contract) VALUES
(seq_furnizori.NEXTVAL, 'Tech Distribution SRL', 'contact@techdist.ro', '0722111222', TO_DATE('10-
01-2023', 'DD-MM-YYYY'));
INSERT INTO FURNIZORI (id_furnizor, denumire, email, telefon, data_contract) VALUES
(seq_furnizori.NEXTVAL, 'Global Electro Import', 'sales@globalelectro.com', '0733444555',
TO_DATE('15-03-2023', 'DD-MM-YYYY'));
INSERT INTO FURNIZORI (id_furnizor, denumire, email, telefon, data_contract) VALUES
(seq_furnizori.NEXTVAL, 'IT Solutions West', 'office@itsolutions.ro', '0744777888', TO_DATE('20-05-
2023', 'DD-MM-YYYY'));
INSERT INTO FURNIZORI (id_furnizor, denumire, email, telefon, data_contract) VALUES
(seq_furnizori.NEXTVAL, 'Home & Deco Logistics', 'livrari@homedeco.ro', '0755999000',
TO_DATE('01-08-2023', 'DD-MM-YYYY'));
INSERT INTO FURNIZORI (id_furnizor, denumire, email, telefon, data_contract) VALUES
(seq_furnizori.NEXTVAL, 'Smart Gadgets Hub', 'partners@smartgadgets.ro', '0766123123',
TO_DATE('10-11-2023', 'DD-MM-YYYY'));

-- CLIENTI (ID-urile generate vor fi 1001 ... 1005)
INSERT INTO CLIENTI (id_client, nume, prenume, email, telefon, data_inregistrare) VALUES
(seq_clienti.NEXTVAL, 'Popescu', 'Andrei', 'andrei.popescu90@gmail.com', '0720123456',
TO_DATE('01-02-2024', 'DD-MM-YYYY'));
INSERT INTO CLIENTI (id_client, nume, prenume, email, telefon, data_inregistrare) VALUES
(seq_clienti.NEXTVAL, 'Ionescu', 'Maria', 'maria.ionescu@yahoo.com', '0730987654', TO_DATE('15-02-
2024', 'DD-MM-YYYY'));
INSERT INTO CLIENTI (id_client, nume, prenume, email, telefon, data_inregistrare) VALUES
(seq_clienti.NEXTVAL, 'Radu', 'Mihai', 'mihai.radu.dev@outlook.com', '0740555666', TO_DATE('20-03-
2024', 'DD-MM-YYYY'));
```

INSERT INTO CLIENTI (id_client, nume, prenume, email, telefon, data_inregistrare) VALUES (seq_clienti.NEXTVAL, 'Dumitrescu', 'Elena', 'elena.d@gmail.com', '0750111222', *TO_DATE*('05-04-2024', 'DD-MM-YYYY'));
INSERT INTO CLIENTI (id_client, nume, prenume, email, telefon, data_inregistrare) VALUES (seq_clienti.NEXTVAL, 'Stan', 'George', 'george.stan88@gmail.com', '0760333444', *TO_DATE*('10-05-2024', 'DD-MM-YYYY'));

-- ANGAJATI (ID-urile generate vor fi 501 ... 505)
INSERT INTO ANGAJATI (id_angajat, nume, prenume, email, telefon, data_angajare) VALUES (seq_angajati.NEXTVAL, 'Marinescu', 'Vlad', 'vlad.marinescu@firma.ro', '0799000111', *TO_DATE*('10-01-2022', 'DD-MM-YYYY'));
INSERT INTO ANGAJATI (id_angajat, nume, prenume, email, telefon, data_angajare) VALUES (seq_angajati.NEXTVAL, 'Gheorghe', 'Ana', 'ana.gheorghe@firma.ro', '0799000222', *TO_DATE*('15-06-2022', 'DD-MM-YYYY'));
INSERT INTO ANGAJATI (id_angajat, nume, prenume, email, telefon, data_angajare) VALUES (seq_angajati.NEXTVAL, 'Dobre', 'Claudiu', 'claudiu.dobre@firma.ro', '0799000333', *TO_DATE*('01-09-2023', 'DD-MM-YYYY'));
INSERT INTO ANGAJATI (id_angajat, nume, prenume, email, telefon, data_angajare) VALUES (seq_angajati.NEXTVAL, 'Nistor', 'Ioana', 'ioana.nistor@firma.ro', '0799000444', *TO_DATE*('20-01-2024', 'DD-MM-YYYY'));
INSERT INTO ANGAJATI (id_angajat, nume, prenume, email, telefon, data_angajare) VALUES (seq_angajati.NEXTVAL, 'Voicu', 'Alexandru', 'alex.voicu@firma.ro', '0799000555', *TO_DATE*('10-03-2024', 'DD-MM-YYYY'));

-- Tabele asociative / dependente

-- PRODUSE_FURNIZORI
-- Declanseaza triggerul 'trg_gestiune_stoc_furnizor'. Folosim ID-urile pe care stim ca secventele le-au generat mai sus.

INSERT INTO PRODUSE_FURNIZORI VALUES (101, 10, *TO_DATE*('01-02-2024', 'DD-MM-YYYY'), NULL, 50, 3200.00);
INSERT INTO PRODUSE_FURNIZORI VALUES (102, 10, *TO_DATE*('01-02-2024', 'DD-MM-YYYY'), NULL, 30, 4500.00);
INSERT INTO PRODUSE_FURNIZORI VALUES (103, 30, *TO_DATE*('05-02-2024', 'DD-MM-YYYY'), NULL, 20, 3800.00);
INSERT INTO PRODUSE_FURNIZORI VALUES (104, 20, *TO_DATE*('10-02-2024', 'DD-MM-YYYY'), NULL, 15, 5100.00);
INSERT INTO PRODUSE_FURNIZORI VALUES (105, 20, *TO_DATE*('12-02-2024', 'DD-MM-YYYY'), NULL, 10, 5000.00);
INSERT INTO PRODUSE_FURNIZORI VALUES (106, 40, *TO_DATE*('15-02-2024', 'DD-MM-YYYY'), NULL, 40, 1600.00);
INSERT INTO PRODUSE_FURNIZORI VALUES (107, 40, *TO_DATE*('15-02-2024', 'DD-MM-YYYY'), NULL, 25, 1400.00);

```
INSERT INTO PRODUSE_FURNIZORI VALUES (108, 50, TO_DATE('20-02-2024', 'DD-MM-YYYY'), NULL, 50, 2200.00);
INSERT INTO PRODUSE_FURNIZORI VALUES (109, 30, TO_DATE('25-02-2024', 'DD-MM-YYYY'), NULL, 100, 300.00);
INSERT INTO PRODUSE_FURNIZORI VALUES (110, 20, TO_DATE('28-02-2024', 'DD-MM-YYYY'), NULL, 30, 600.00);
INSERT INTO PRODUSE_FURNIZORI VALUES (101, 20, TO_DATE('01-03-2024', 'DD-MM-YYYY'), NULL, 20, 3150.00);
INSERT INTO PRODUSE_FURNIZORI VALUES (103, 10, TO_DATE('05-03-2024', 'DD-MM-YYYY'), NULL, 10, 3850.00);


-- COMENZI (ID-urile generate vor fi 2001 ... 2010)
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES (seq_comenzi.NEXTVAL, 1001, TO_DATE('10-03-2024', 'DD-MM-YYYY'), 'anulata', 'card');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES (seq_comenzi.NEXTVAL, 1002, TO_DATE('12-03-2024', 'DD-MM-YYYY'), 'anulata', 'numerar');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES (seq_comenzi.NEXTVAL, 1003, TO_DATE('20-03-2024', 'DD-MM-YYYY'), 'in asteptare', 'card');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES (seq_comenzi.NEXTVAL, 1001, TO_DATE('20-03-2024', 'DD-MM-YYYY'), 'in asteptare', 'card');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES (seq_comenzi.NEXTVAL, 1004, TO_DATE('05-04-2024', 'DD-MM-YYYY'), 'anulata', 'card');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES (seq_comenzi.NEXTVAL, 1005, TO_DATE('10-05-2024', 'DD-MM-YYYY'), 'in asteptare', 'numerar');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES (seq_comenzi.NEXTVAL, 1002, TO_DATE('05-04-2024', 'DD-MM-YYYY'), 'in asteptare', 'card');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES (seq_comenzi.NEXTVAL, 1003, TO_DATE('10-04-2024', 'DD-MM-YYYY'), 'in asteptare', 'card');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES (seq_comenzi.NEXTVAL, 1005, TO_DATE('10-05-2024', 'DD-MM-YYYY'), 'in asteptare', 'numerar');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES (seq_comenzi.NEXTVAL, 1001, TO_DATE('15-04-2024', 'DD-MM-YYYY'), 'in asteptare', 'card');


-- DETALII_COMANDA
-- Declanseaza triggerul 'trg_gestiune_stoc_vanzari'. Folosim ID-urile generate de secvente mai sus.

-- Comanda 2001
INSERT INTO DETALII_COMANDA VALUES (2001, 101, 1, 3999.99);
INSERT INTO DETALII_COMANDA VALUES (2001, 109, 1, 450.00);

-- Comanda 2002
INSERT INTO DETALII_COMANDA VALUES (2002, 106, 1, 2100.00);

-- Comanda 2003
INSERT INTO DETALII_COMANDA VALUES (2003, 108, 1, 2699.99);
```

```
INSERT INTO DETALII_COMANDA VALUES (2003, 105, 1, 6200.00);

-- Comanda 2004
INSERT INTO DETALII_COMANDA VALUES (2004, 104, 1, 5999.00);

-- Comanda 2005
INSERT INTO DETALII_COMANDA VALUES (2005, 107, 1, 1850.00);

-- Comanda 2006
INSERT INTO DETALII_COMANDA VALUES (2006, 102, 1, 5400.00);

-- Comanda 2007
INSERT INTO DETALII_COMANDA VALUES (2007, 110, 2, 899.00);

-- Comanda 2008
INSERT INTO DETALII_COMANDA VALUES (2008, 103, 1, 4500.50);

-- Comanda 2009
INSERT INTO DETALII_COMANDA VALUES (2009, 109, 2, 450.00);

-- Comanda 2010
INSERT INTO DETALII_COMANDA VALUES (2010, 101, 1, 3999.99);


-- FACTURI
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES
(seq_facturi.NEXTVAL, 2001, TO_DATE('10-03-2024', 'DD-MM-YYYY'), 'platita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES
(seq_facturi.NEXTVAL, 2002, TO_DATE('12-03-2024', 'DD-MM-YYYY'), 'platita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES
(seq_facturi.NEXTVAL, 2003, TO_DATE('20-03-2024', 'DD-MM-YYYY'), 'platita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES
(seq_facturi.NEXTVAL, 2004, TO_DATE('20-03-2024', 'DD-MM-YYYY'), 'platita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES
(seq_facturi.NEXTVAL, 2005, TO_DATE('05-04-2024', 'DD-MM-YYYY'), 'neplatita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES
(seq_facturi.NEXTVAL, 2006, TO_DATE('10-05-2024', 'DD-MM-YYYY'), 'platita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES
(seq_facturi.NEXTVAL, 2007, TO_DATE('05-06-2024', 'DD-MM-YYYY'), 'neplatita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES
(seq_facturi.NEXTVAL, 2008, TO_DATE('10-06-2024', 'DD-MM-YYYY'), 'platita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES
(seq_facturi.NEXTVAL, 2009, TO_DATE('12-06-2024', 'DD-MM-YYYY'), 'neplatita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES
(seq_facturi.NEXTVAL, 2010, TO_DATE('15-06-2024', 'DD-MM-YYYY'), 'platita');
```

-- PROCESARE_COMENZI
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES (501, 2001, *TO_DATE*('10-03-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES (502, 2002, *TO_DATE*('12-03-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES (501, 2003, *TO_DATE*('15-06-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES (503, 2004, *TO_DATE*('20-06-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES (502, 2005, *TO_DATE*('25-06-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES (504, 2006, *TO_DATE*('01-06-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES (503, 2007, *TO_DATE*('05-06-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES (501, 2008, *TO_DATE*('10-06-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES (505, 2009, *TO_DATE*('12-06-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES (504, 2010, *TO_DATE*('15-06-2024', 'DD-MM-YYYY'));

COMMIT;

```
393  -- Cerinta 5
394
395  -- SECVENTE
396  -- Stergem secventele vechi daca exista
397  BEGIN
398    EXECUTE IMMEDIATE 'DROP SEQUENCE seq_categorii';
399    EXECUTE IMMEDIATE 'DROP SEQUENCE seq_produse';
400    EXECUTE IMMEDIATE 'DROP SEQUENCE seq_furnizori';
401    EXECUTE IMMEDIATE 'DROP SEQUENCE seq_clienti';
402    EXECUTE IMMEDIATE 'DROP SEQUENCE seq_angajati';
403    EXECUTE IMMEDIATE 'DROP SEQUENCE seq_comenzi';
404    EXECUTE IMMEDIATE 'DROP SEQUENCE seq_facturi';
405  EXCEPTION
406    WHEN OTHERS THEN NULL; -- Ignora eroarea daca secventele nu exista
407  END;
408  /
409
410  -- Creare Secvente
411  CREATE SEQUENCE seq_categorii START WITH 1 INCREMENT BY 1;
412  CREATE SEQUENCE seq_produse   START WITH 101 INCREMENT BY 1;
413  CREATE SEQUENCE seq_furnizori START WITH 10 INCREMENT BY 10; -- Furnizorii vor fi 10, 20, 30...
414  CREATE SEQUENCE seq_clienti   START WITH 1001 INCREMENT BY 1;
415  CREATE SEQUENCE seq_angajati  START WITH 501 INCREMENT BY 1;
416  CREATE SEQUENCE seq_comenzi   START WITH 2001 INCREMENT BY 1;
417  CREATE SEQUENCE seq_facturi   START WITH 3001 INCREMENT BY 1;
418
419
420  -- Tabele independente
421
422
423  -- CATEGORII (ID-urile generate vor fi 1, 2, 3, 4, 5)
424  INSERT INTO CATEGORII (id_categorie, nume, cod) VALUES ( id_categorie seq_categorii.NEXTVAL,  nume 'Telefoane Mobile',  cod 'TEL');
425  INSERT INTO CATEGORII (id_categorie, nume, cod) VALUES ( id_categorie seq_categorii.NEXTVAL,  nume 'Laptopuri & IT',  cod 'LAP');
426  INSERT INTO CATEGORII (id_categorie, nume, cod) VALUES ( id_categorie seq_categorii.NEXTVAL,  nume 'Televizoare & Audio',  cod 'TV');
427  INSERT INTO CATEGORII (id_categorie, nume, cod) VALUES ( id_categorie seq_categorii.NEXTVAL,  nume 'Electrocasnice Mici',  cod 'ELECTRO');
428  INSERT INTO CATEGORII (id_categorie, nume, cod) VALUES ( id_categorie seq_categorii.NEXTVAL,  nume 'Gaming',  cod 'GAME');
```

```
-- PRODUSE (ID-urile generate vor fi 101, 102 ... 110)
-- la id_categorie folosim valorile 1-5 generate mai sus.
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES ( id_produs seq_produse.NEXTVAL,  id_categorie 1,  denumire 'Samsung Galaxy S23',  pret 3999.99,  stoc 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES ( id_produs seq_produse.NEXTVAL,  id_categorie 1,  denumire 'iPhone 15 Pro',  pret 5400.00,  stoc 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES ( id_produs seq_produse.NEXTVAL,  id_categorie 2,  denumire 'Laptop ASUS TUF Gaming',  pret 4500.50,  stoc 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES ( id_produs seq_produse.NEXTVAL,  id_categorie 2,  denumire 'MacBook Air M2',  pret 5999.00,  stoc 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES ( id_produs seq_produse.NEXTVAL,  id_categorie 3,  denumire 'Televizor LG OLED',  pret 6200.00,  stoc 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES ( id_produs seq_produse.NEXTVAL,  id_categorie 4,  denumire 'Espresor Philips LatteGo',  pret 2100.00,  stoc 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES ( id_produs seq_produse.NEXTVAL,  id_categorie 4,  denumire 'Aspirator Robot Roborock',  pret 1850.00,  stoc 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES ( id_produs seq_produse.NEXTVAL,  id_categorie 5,  denumire 'Consola PlayStation 5',  pret 2699.99,  stoc 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES ( id_produs seq_produse.NEXTVAL,  id_categorie 2,  denumire 'Mouse Logitech MX Master',  pret 450.00,  stoc 0);
INSERT INTO PRODUSE (id_produs, id_categorie, denumire, pret, stoc) VALUES ( id_produs seq_produse.NEXTVAL,  id_categorie 3,  denumire 'Soundbar Samsung',  pret 899.00,  stoc 0);

-- FURNIZORI (ID-urile generate vor fi 10, 20, 30, 40, 50)
INSERT INTO FURNIZORI (id_furnizor, denumire, email, telefon, data_contract) VALUES ( id_furnizor seq_furnizori.NEXTVAL,  denumire 'Tech Distribution SRL',  email 'contact@techdist.ro',  telefon '0722111222',  data_contract TO_DATE('10-01-2023', 'DD-MM-YYYY'));
INSERT INTO FURNIZORI (id_furnizor, denumire, email, telefon, data_contract) VALUES ( id_furnizor seq_furnizori.NEXTVAL,  denumire 'Global Electro Import',  email 'sales@globalelectro.com',  telefon '0733444555',  data_contract TO_DATE('15-03-2023', 'DD-MM-YYYY'));
INSERT INTO FURNIZORI (id_furnizor, denumire, email, telefon, data_contract) VALUES ( id_furnizor seq_furnizori.NEXTVAL,  denumire 'IT Solutions West',  email 'office@itsolutions.ro',  telefon '0744777888',  data_contract TO_DATE('20-05-2023', 'DD-MM-YYYY'));
INSERT INTO FURNIZORI (id_furnizor, denumire, email, telefon, data_contract) VALUES ( id_furnizor seq_furnizori.NEXTVAL,  denumire 'Home & Deco Logistics',  email 'livrari@homedeco.ro',  telefon '0755999000',  data_contract TO_DATE('01-08-2023', 'DD-MM-YYYY'));
INSERT INTO FURNIZORI (id_furnizor, denumire, email, telefon, data_contract) VALUES ( id_furnizor seq_furnizori.NEXTVAL,  denumire 'Smart Gadgets Hub',  email 'partners@smartgadgets.ro',  telefon '0766123123',  data_contract TO_DATE('10-11-2023', 'DD-MM-YYYY'));

-- CLIENTI (ID-urile generate vor fi 1001 ... 1005)
INSERT INTO CLIENTI (id_client, nume, prenume, email, telefon, data_inregistrare) VALUES ( id_client seq_clienti.NEXTVAL,  nume 'Popescu',  prenume 'Andrei',  email 'andrei.popescu90@gmail.com',  telefon '0720123456',  data_inregistrare TO_DATE('01-02-2024', 'DD-MM-YYYY
INSERT INTO CLIENTI (id_client, nume, prenume, email, telefon, data_inregistrare) VALUES ( id_client seq_clienti.NEXTVAL,  nume 'Ionescu',  prenume 'Maria',  email 'maria.ionescu@yahoo.com',  telefon '0730987654',  data_inregistrare TO_DATE('15-02-2024', 'DD-MM-YYYY'));
INSERT INTO CLIENTI (id_client, nume, prenume, email, telefon, data_inregistrare) VALUES ( id_client seq_clienti.NEXTVAL,  nume 'Radu',  prenume 'Mihai',  email 'mihai.radu.dev@outlook.com',  telefon '0740555666',  data_inregistrare TO_DATE('20-03-2024', 'DD-MM-YYYY'));
INSERT INTO CLIENTI (id_client, nume, prenume, email, telefon, data_inregistrare) VALUES ( id_client seq_clienti.NEXTVAL,  nume 'Dumitrescu',  prenume 'Elena',  email 'elena.d@gmail.com',  telefon '0750111222',  data_inregistrare TO_DATE('05-04-2024', 'DD-MM-YYYY'));
INSERT INTO CLIENTI (id_client, nume, prenume, email, telefon, data_inregistrare) VALUES ( id_client seq_clienti.NEXTVAL,  nume 'Stan',  prenume 'George',  email 'george.stan88@gmail.com',  telefon '0760333444',  data_inregistrare TO_DATE('10-05-2024', 'DD-MM-YYYY'));

-- ANGAJATI (ID-urile generate vor fi 501 ... 505)
INSERT INTO ANGAJATI (id_angajat, nume, prenume, email, telefon, data_angajare) VALUES ( id_angajat seq_angajati.NEXTVAL,  nume 'Marinescu',  prenume 'Vlad',  email 'vlad.marinescu@firma.ro',  telefon '0799000111',  data_angajare TO_DATE('10-01-2022', 'DD-MM-YYYY'));
INSERT INTO ANGAJATI (id_angajat, nume, prenume, email, telefon, data_angajare) VALUES ( id_angajat seq_angajati.NEXTVAL,  nume 'Gheorghe',  prenume 'Ana',  email 'ana.gheorghe@firma.ro',  telefon '0799000222',  data_angajare TO_DATE('15-06-2022', 'DD-MM-YYYY'));
INSERT INTO ANGAJATI (id_angajat, nume, prenume, email, telefon, data_angajare) VALUES ( id_angajat seq_angajati.NEXTVAL,  nume 'Dobre',  prenume 'Claudiu',  email 'claudiu.dobre@firma.ro',  telefon '0799000333',  data_angajare TO_DATE('01-09-2023', 'DD-MM-YYYY'));
INSERT INTO ANGAJATI (id_angajat, nume, prenume, email, telefon, data_angajare) VALUES ( id_angajat seq_angajati.NEXTVAL,  nume 'Nistor',  prenume 'Ioana',  email 'ioana.nistor@firma.ro',  telefon '0799000444',  data_angajare TO_DATE('20-01-2024', 'DD-MM-YYYY'));
INSERT INTO ANGAJATI (id_angajat, nume, prenume, email, telefon, data_angajare) VALUES ( id_angajat seq_angajati.NEXTVAL,  nume 'Voicu',  prenume 'Alexandru',  email 'alex.voicu@firma.ro',  telefon '0799000555',  data_angajare TO_DATE('10-03-2024', 'DD-MM-YYYY'));
```

```
-- Tabele asociative / dependente


-- PRODUSE_FURNIZORI
-- Declanseaza triggerul 'trg_gestiune_stoc_furnizor'. Folosim ID-urile pe care stim ca secventele le-au generat mai sus.

INSERT INTO PRODUSE_FURNIZORI VALUES ( id_produs 101,  id_furnizor 10,  data_inceput TO_DATE('01-02-2024', 'DD-MM-YYYY'),  data_sfarsit NULL,  stoc_achizitie 50,  pret_achizitie 3200.00);
INSERT INTO PRODUSE_FURNIZORI VALUES ( id_produs 102,  id_furnizor 10,  data_inceput TO_DATE('01-02-2024', 'DD-MM-YYYY'),  data_sfarsit NULL,  stoc_achizitie 30,  pret_achizitie 4500.00);
INSERT INTO PRODUSE_FURNIZORI VALUES ( id_produs 103,  id_furnizor 30,  data_inceput TO_DATE('05-02-2024', 'DD-MM-YYYY'),  data_sfarsit NULL,  stoc_achizitie 20,  pret_achizitie 3800.00);
INSERT INTO PRODUSE_FURNIZORI VALUES ( id_produs 104,  id_furnizor 20,  data_inceput TO_DATE('10-02-2024', 'DD-MM-YYYY'),  data_sfarsit NULL,  stoc_achizitie 15,  pret_achizitie 5100.00);
INSERT INTO PRODUSE_FURNIZORI VALUES ( id_produs 105,  id_furnizor 20,  data_inceput TO_DATE('12-02-2024', 'DD-MM-YYYY'),  data_sfarsit NULL,  stoc_achizitie 10,  pret_achizitie 5000.00);
INSERT INTO PRODUSE_FURNIZORI VALUES ( id_produs 106,  id_furnizor 40,  data_inceput TO_DATE('15-02-2024', 'DD-MM-YYYY'),  data_sfarsit NULL,  stoc_achizitie 40,  pret_achizitie 1600.00);
INSERT INTO PRODUSE_FURNIZORI VALUES ( id_produs 107,  id_furnizor 40,  data_inceput TO_DATE('15-02-2024', 'DD-MM-YYYY'),  data_sfarsit NULL,  stoc_achizitie 25,  pret_achizitie 1400.00);
INSERT INTO PRODUSE_FURNIZORI VALUES ( id_produs 108,  id_furnizor 50,  data_inceput TO_DATE('20-02-2024', 'DD-MM-YYYY'),  data_sfarsit NULL,  stoc_achizitie 50,  pret_achizitie 2200.00);
INSERT INTO PRODUSE_FURNIZORI VALUES ( id_produs 109,  id_furnizor 30,  data_inceput TO_DATE('25-02-2024', 'DD-MM-YYYY'),  data_sfarsit NULL,  stoc_achizitie 100,  pret_achizitie 300.00);
INSERT INTO PRODUSE_FURNIZORI VALUES ( id_produs 110,  id_furnizor 20,  data_inceput TO_DATE('28-02-2024', 'DD-MM-YYYY'),  data_sfarsit NULL,  stoc_achizitie 30,  pret_achizitie 600.00);
INSERT INTO PRODUSE_FURNIZORI VALUES ( id_produs 101,  id_furnizor 20,  data_inceput TO_DATE('01-03-2024', 'DD-MM-YYYY'),  data_sfarsit NULL,  stoc_achizitie 20,  pret_achizitie 3150.00);
INSERT INTO PRODUSE_FURNIZORI VALUES ( id_produs 103,  id_furnizor 10,  data_inceput TO_DATE('05-03-2024', 'DD-MM-YYYY'),  data_sfarsit NULL,  stoc_achizitie 10,  pret_achizitie 3850.00);


-- COMENZI (ID-urile generate vor fi 2001 ... 2010)
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES ( id_comanda seq_comenzi.NEXTVAL,  id_client 1001,  data_comanda TO_DATE('10-03-2024', 'DD-MM-YYYY'),  status 'anulata',  metoda_plata 'card');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES ( id_comanda seq_comenzi.NEXTVAL,  id_client 1002,  data_comanda TO_DATE('12-03-2024', 'DD-MM-YYYY'),  status 'anulata',  metoda_plata 'numerar');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES ( id_comanda seq_comenzi.NEXTVAL,  id_client 1003,  data_comanda TO_DATE('20-03-2024', 'DD-MM-YYYY'),  status 'in asteptare',  metoda_plata 'card');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES ( id_comanda seq_comenzi.NEXTVAL,  id_client 1001,  data_comanda TO_DATE('20-03-2024', 'DD-MM-YYYY'),  status 'in asteptare',  metoda_plata 'card');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES ( id_comanda seq_comenzi.NEXTVAL,  id_client 1004,  data_comanda TO_DATE('05-04-2024', 'DD-MM-YYYY'),  status 'anulata',  metoda_plata 'card');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES ( id_comanda seq_comenzi.NEXTVAL,  id_client 1005,  data_comanda TO_DATE('10-05-2024', 'DD-MM-YYYY'),  status 'in asteptare',  metoda_plata 'numerar');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES ( id_comanda seq_comenzi.NEXTVAL,  id_client 1002,  data_comanda TO_DATE('05-04-2024', 'DD-MM-YYYY'),  status 'in asteptare',  metoda_plata 'card');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES ( id_comanda seq_comenzi.NEXTVAL,  id_client 1003,  data_comanda TO_DATE('10-04-2024', 'DD-MM-YYYY'),  status 'in asteptare',  metoda_plata 'card');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES ( id_comanda seq_comenzi.NEXTVAL,  id_client 1005,  data_comanda TO_DATE('10-05-2024', 'DD-MM-YYYY'),  status 'in asteptare',  metoda_plata 'numerar');
INSERT INTO COMENZI (id_comanda, id_client, data_comanda, status, metoda_plata) VALUES ( id_comanda seq_comenzi.NEXTVAL,  id_client 1001,  data_comanda TO_DATE('15-04-2024', 'DD-MM-YYYY'),  status 'in asteptare',  metoda_plata 'card');
```

```oracle_plsql
-- DETALII_COMANDA
-- Declanseaza triggerul 'trg_gestiune_stoc_vanzari'. Folosim ID-urile generate de secvente mai sus.

-- Comanda 2001
INSERT INTO DETALII_COMANDA VALUES ( id_comanda 2001, id_produs 101, cantitate 1, pret_unitar 3999.99);
INSERT INTO DETALII_COMANDA VALUES ( id_comanda 2001, id_produs 109, cantitate 1, pret_unitar 450.00);

-- Comanda 2002
INSERT INTO DETALII_COMANDA VALUES ( id_comanda 2002, id_produs 106, cantitate 1, pret_unitar 2100.00);

-- Comanda 2003
INSERT INTO DETALII_COMANDA VALUES ( id_comanda 2003, id_produs 108, cantitate 1, pret_unitar 2699.99);
INSERT INTO DETALII_COMANDA VALUES ( id_comanda 2003, id_produs 105, cantitate 1, pret_unitar 6200.00);

-- Comanda 2004
INSERT INTO DETALII_COMANDA VALUES ( id_comanda 2004, id_produs 104, cantitate 1, pret_unitar 5999.00);

-- Comanda 2005
INSERT INTO DETALII_COMANDA VALUES ( id_comanda 2005, id_produs 107, cantitate 1, pret_unitar 1850.00);

-- Comanda 2006
INSERT INTO DETALII_COMANDA VALUES ( id_comanda 2006, id_produs 102, cantitate 1, pret_unitar 5400.00);

-- Comanda 2007
INSERT INTO DETALII_COMANDA VALUES ( id_comanda 2007, id_produs 110, cantitate 2, pret_unitar 899.00);

-- Comanda 2008
INSERT INTO DETALII_COMANDA VALUES ( id_comanda 2008, id_produs 103, cantitate 1, pret_unitar 4500.50);

-- Comanda 2009
INSERT INTO DETALII_COMANDA VALUES ( id_comanda 2009, id_produs 109, cantitate 2, pret_unitar 450.00);

-- Comanda 2010
INSERT INTO DETALII_COMANDA VALUES ( id_comanda 2010, id_produs 101, cantitate 1, pret_unitar 3999.99);
```

```oracle_plsql
-- FACTURI
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES ( id_factura seq_facturi.NEXTVAL, id_comanda 2001, data_emitere TO_DATE('10-03-2024', 'DD-MM-YYYY'), status_plata 'platita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES ( id_factura seq_facturi.NEXTVAL, id_comanda 2002, data_emitere TO_DATE('12-03-2024', 'DD-MM-YYYY'), status_plata 'platita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES ( id_factura seq_facturi.NEXTVAL, id_comanda 2003, data_emitere TO_DATE('20-03-2024', 'DD-MM-YYYY'), status_plata 'platita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES ( id_factura seq_facturi.NEXTVAL, id_comanda 2004, data_emitere TO_DATE('20-03-2024', 'DD-MM-YYYY'), status_plata 'platita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES ( id_factura seq_facturi.NEXTVAL, id_comanda 2005, data_emitere TO_DATE('05-04-2024', 'DD-MM-YYYY'), status_plata 'neplatita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES ( id_factura seq_facturi.NEXTVAL, id_comanda 2006, data_emitere TO_DATE('10-05-2024', 'DD-MM-YYYY'), status_plata 'platita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES ( id_factura seq_facturi.NEXTVAL, id_comanda 2007, data_emitere TO_DATE('05-06-2024', 'DD-MM-YYYY'), status_plata 'neplatita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES ( id_factura seq_facturi.NEXTVAL, id_comanda 2008, data_emitere TO_DATE('10-06-2024', 'DD-MM-YYYY'), status_plata 'platita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES ( id_factura seq_facturi.NEXTVAL, id_comanda 2009, data_emitere TO_DATE('12-06-2024', 'DD-MM-YYYY'), status_plata 'neplatita');
INSERT INTO FACTURI (id_factura, id_comanda, data_emitere, status_plata) VALUES ( id_factura seq_facturi.NEXTVAL, id_comanda 2010, data_emitere TO_DATE('15-06-2024', 'DD-MM-YYYY'), status_plata 'platita');


-- PROCESARE_COMENZI
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES ( id_angajat 501, id_comanda 2001, data_proc TO_DATE('10-03-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES ( id_angajat 502, id_comanda 2002, data_proc TO_DATE('12-03-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES ( id_angajat 501, id_comanda 2003, data_proc TO_DATE('15-06-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES ( id_angajat 503, id_comanda 2004, data_proc TO_DATE('20-06-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES ( id_angajat 502, id_comanda 2005, data_proc TO_DATE('25-06-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES ( id_angajat 504, id_comanda 2006, data_proc TO_DATE('01-06-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES ( id_angajat 503, id_comanda 2007, data_proc TO_DATE('05-06-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES ( id_angajat 501, id_comanda 2008, data_proc TO_DATE('10-06-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES ( id_angajat 505, id_comanda 2009, data_proc TO_DATE('12-06-2024', 'DD-MM-YYYY'));
INSERT INTO PROCESARE_COMENZI (id_angajat, id_comanda, data_proc) VALUES ( id_angajat 504, id_comanda 2010, data_proc TO_DATE('15-06-2024', 'DD-MM-YYYY'));

COMMIT;
```

console_1 [@fabianmiu15] console [@localhost] console [db1_grupa241] console [Magazin Online] ANGAJATI [Magazin Online]

WHERE ORDER BY

	ID_ANGAJAT	NUME	PRENUME	EMAIL	TELEFON	DATA_ANGAJARE
1	501	Marinescu	Vlad	vlad.marinescu@firma.ro	0799000111	2022-01-10
2	502	Gheorghe	Ana	ana.gheorghe@firma.ro	0799000222	2022-06-15
3	503	Dobre	Claudiu	claudiu.dobre@firma.ro	0799000333	2023-09-01
4	504	Nistor	Ioana	ioana.nistor@firma.ro	0799000444	2024-01-20
5	505	Voicu	Alexandru	alex.voicu@firma.ro	0799000555	2024-03-10

console_1 [@fabianmiu15] console [@localhost] console [db1_grupa241] console [Magazin Online] CATEGORII [Magazin Online]

WHERE ORDER BY

	ID_CATEGORIE	NUME	COD
1	1	Telefoane Mobile	TEL
2	2	Laptopuri & IT	LAP
3	3	Televizoare & Audio	TV
4	4	Electrocasnice Mici	ELECTRO
5	5	Gaming	GAME

console_1 [@fabianmiu15] console [@localhost] console [db1_grupa241] console [Magazin Online] CLIENTI [Magazin Online]

WHERE ORDER BY

	ID_CLIENT	NUME	PRENUME	EMAIL	TELEFON	DATA_INREGISTRARE
1	1001	Popescu	Andrei	andrei.popescu90@gmail.com	0720123456	2024-02-01
2	1002	Ionescu	Maria	maria.ionescu@yahoo.com	0730987654	2024-02-15
3	1003	Radu	Mihai	mihai.radu.dev@outlook.com	0740555666	2024-03-20
4	1004	Dumitrescu	Elena	elena.d@gmail.com	0750111222	2024-04-05
5	1005	Stan	George	george.stan88@gmail.com	0760333444	2024-05-10

console_1 [@fabianmiu15] console [@localhost] console [db1_grupa241] console [Magazin Online] COMENZI [Magazin Online]

WHERE ORDER BY

	ID_COMANDA	ID_CLIENT	DATA_COMANDA	STATUS	METODA_PLATA
1	2001	1001	2024-03-10	anulata	card
2	2002	1002	2024-03-12	anulata	numerar
3	2003	1003	2024-03-20	in asteptare	card
4	2004	1001	2024-03-20	in asteptare	card
5	2005	1004	2024-04-05	anulata	card
6	2006	1005	2024-05-10	in asteptare	numerar
7	2007	1002	2024-04-05	in asteptare	card
8	2008	1003	2024-04-10	in asteptare	card
9	2009	1005	2024-05-10	in asteptare	numerar
10	2010	1001	2024-04-15	in asteptare	card

console_1 [@fabianmiu15] console [@localhost] console [db1_grupa241] console [Magazin Online] COMENZI [Magazin Online] DETALII_COMANDA [Magazin Online]

Tx: Manual DDL

WHERE ORDER BY

	ID_COMANDA	ID_PRODUS	CANTITATE	PRET_UNITAR
1	2001	101	1	3999.99
2	2001	109	1	450.00
3	2002	106	1	2100.00
4	2003	108	1	2699.99
5	2003	105	1	6200.00
6	2004	104	1	5999.00
7	2005	107	1	1850.00
8	2006	102	1	5400.00
9	2007	110	2	899.00
10	2008	103	1	4500.50
11	2009	109	2	450.00
12	2010	101	1	3999.99

console_1 [@fabianmiu15] console [@localhost] console [db1_grupa241] console [Magazin Online] FACTURI [Magazin Online]

Tx: Manual DDL

WHERE ORDER BY

	ID_FACTURA	ID_COMANDA	DATA_EMITERE	STATUS_PLATA
1	3001	2001	2024-03-10	platita
2	3002	2002	2024-03-12	platita
3	3003	2003	2024-03-20	platita
4	3004	2004	2024-03-20	platita
5	3005	2005	2024-04-05	neplatita
6	3006	2006	2024-05-10	platita
7	3007	2007	2024-06-05	neplatita
8	3008	2008	2024-06-10	platita
9	3009	2009	2024-06-12	neplatita
10	3010	2010	2024-06-15	platita

console_1 [@fabianmiu15] console [@localhost] console [db1_grupa241] console [Magazin Online] FURNIZORI [Magazin Online]

Tx: Manual DDL

WHERE ORDER BY

	ID_FURNIZOR	DENUMIRE	EMAIL	TELEFON	DATA_CONTRACT
1	10	Tech Distribution SRL	contact@techdist.ro	0722111222	2023-01-10
2	20	Global Electro Import	sales@globalelectro.com	0733444555	2023-03-15
3	30	IT Solutions West	office@itsolutions.ro	0744777888	2023-05-20
4	40	Home & Deco Logistics	livrari@homedeco.ro	0755999000	2023-08-01
5	50	Smart Gadgets Hub	partners@smartgadgets.ro	0766123123	2023-11-10

console_1 [@fabianmiu15] console [@localhost] console [db1_grupa241] console [Magazin Online] PROCESARE_COMENZI [Magazin Online]

Tx: Manual DDL

WHERE ORDER BY

	ID_ANGAJAT	ID_COMANDA	DATA_PROC
1	501	2001	2024-03-10
2	502	2002	2024-03-12
3	501	2003	2024-06-15
4	503	2004	2024-06-20
5	502	2005	2024-06-25
6	504	2006	2024-06-01
7	503	2007	2024-06-05
8	501	2008	2024-06-10
9	505	2009	2024-06-12
10	504	2010	2024-06-15

console_1 [@fabianmiu15] console [@localhost] console [db1_grupa241] console [Magazin Online] PRODUSE [Magazin Online]

Tx: Manual DDL

WHERE ORDER BY

	ID_PRODUS	ID_CATEGORIE	DENUMIRE	PRET	STOC
1	101	1	Samsung Galaxy S23	3999.99	68
2	102	1	iPhone 15 Pro	5400.00	29
3	103	2	Laptop ASUS TUF Gaming	4500.50	29
4	104	2	MacBook Air M2	5999.00	14
5	105	3	Televizor LG OLED	6200.00	9
6	106	4	Espresor Philips LatteGo	2100.00	39
7	107	4	Aspirator Robot Roborock	1850.00	24
8	108	5	Consola PlayStation 5	2699.99	49
9	109	2	Mouse Logitech MX Master	450.00	97
10	110	3	Soundbar Samsung	899.00	28

console_1 [@fabianmiu15] console [@localhost] console [db1_grupa241] console [Magazin Online] PRODUSE_FURNIZORI [Magazin Online]

Tx: Manual DDL

WHERE ORDER BY

	ID_PRODUS	ID_FURNIZOR	DATA_INCEPUT	DATA_SFARSIT	STOC_ACHIZITIE	PRET_ACHIZITIE
1	101	10	2024-02-01	<null>	50	3200.00
2	102	10	2024-02-01	<null>	30	4500.00
3	103	30	2024-02-05	<null>	20	3800.00
4	104	20	2024-02-10	<null>	15	5100.00
5	105	20	2024-02-12	<null>	10	5000.00
6	106	40	2024-02-15	<null>	40	1600.00
7	107	40	2024-02-15	<null>	25	1400.00
8	108	50	2024-02-20	<null>	50	2200.00
9	109	30	2024-02-25	<null>	100	300.00
10	110	20	2024-02-28	<null>	30	600.00
11	101	20	2024-03-01	<null>	20	3150.00
12	103	10	2024-03-05	<null>	10	3850.00

6. **Formularea în limbaj natural a unei probleme pe care să o rezolvăm folosind un subprogram stocat independent care să utilizeze toate cele 3 tipuri de colecții studiate și apelarea acestuia**

Determinați cât au cheltuit în total clienții care au cumpărat produse din 3 categorii specifice, numite „categorii premium”.

a) Definiți o listă fixă cu 3 „categorii premium” pentru care se va face analiza.
b) Identificați toți clienții care au cumpărat produse ce fac parte din aceste categorii țintă.
c) Pentru fiecare client identificat în lista de mai sus, calculați valoarea totală a tuturor comenzilor sale.

```
CREATE OR REPLACE PROCEDURE raport_vanzari_categorii IS
   -- a) Definim lista „categoriilor premium” (dimensiune fixa => Varray)
   TYPE t_vector_categorii IS VARRAY(3) OF VARCHAR2(50);
   v_categorii_tinta t_vector_categorii := t_vector_categorii('Laptopuri & IT', 'Telefoane Mobile',
'Gaming');

   -- b) Lista ID-urilor clientilor gasiti (nu stim cati vor fi gasiti => Nested Table)
   TYPE t_lista_clienti IS TABLE OF NUMBER;
   v_clienti_gasiti t_lista_clienti := t_lista_clienti();

   -- Asocierea ID_Client -> Suma Totala (Index-By Table)
   TYPE t_statistica_vanzari IS TABLE OF NUMBER INDEX BY BINARY_INTEGER;
   v_totaluri_clienti t_statistica_vanzari;

   -- Variabile auxiliare
   v_total_temp NUMBER;
   v_idx       BINARY_INTEGER; -- Indexul pentru parcurgere
   v_cat_curent VARCHAR2(50);
BEGIN

   -- Populam Nested Table
   FOR i IN 1..v_categorii_tinta.COUNT LOOP
      v_cat_curent := v_categorii_tinta(i);

      FOR r_client IN ( -- selecteaza toti clientii care au cumparat produse din v_cat_curent
         SELECT DISTINCT c.id_client
         FROM CLIENTI c, COMENZI com, DETALII_COMANDA dc, PRODUSE p, CATEGORII cat
         WHERE c.id_client = com.id_client
           AND com.id_comanda = dc.id_comanda
           AND dc.id_produs = p.id_produs
           AND p.id_categorie = cat.id_categorie
           AND cat.nume = v_cat_curent
      ) LOOP
         v_clienti_gasiti.EXTEND;
```

```
        v_clienti_gasiti(v_clienti_gasiti.LAST) := r_client.id_client;
      END LOOP;
    END LOOP;

    -- Verificam daca am gasit clienti
    IF v_clienti_gasiti.COUNT = 0 THEN
      DBMS_OUTPUT.PUT_LINE('Nu au fost gssiti clienti.');
      RETURN;
    END IF;

    -- Populam Index-By Table
    FOR i IN v_clienti_gasiti.FIRST .. v_clienti_gasiti.LAST LOOP

      SELECT NVL(SUM(dc.cantitate * dc.pret_unitar), 0)
      INTO v_total_temp
      FROM COMENZI com, DETALII_COMANDA dc
      WHERE com.id_comanda = dc.id_comanda
        AND com.id_client = v_clienti_gasiti(i);

      v_totaluri_clienti(v_clienti_gasiti(i)) := v_total_temp;
    END LOOP;

    -- Afisare
    DBMS_OUTPUT.PUT_LINE('--- Clientii Premium ---');
    v_idx := v_totaluri_clienti.FIRST;
    WHILE v_idx IS NOT NULL LOOP
      DBMS_OUTPUT.PUT_LINE('Client ID: ' || v_idx ||
                           ' | Valoare Totala Achizitii: ' || v_totaluri_clienti(v_idx) || ' RON');
      v_idx := v_totaluri_clienti.NEXT(v_idx);
    END LOOP;
-- Ordinea clientilor este garantata crescator dupa ID deoarece Index-By Table cu cheie numerica
stocheaza elementele sortate automat dupa index
END;
/

BEGIN
  raport_vanzari_categorii;
END;
/
```

```oracle_plsql
-- Cerinta 6


-- Determinati cat au cheltuit in total clientii care au cumparat produse din 3 categorii specifice, numite „categorii premium".
-- a) Definiti o lista fixa cu 3 categorii „premium" pentru care se va face analiza.
-- b) Identificati toti clientii care au cumparat produse ce fac parte din aceste categorii tinta.
-- c) Pentru fiecare client identificat in lista de mai sus, calculati valoarea totala a tuturor comenzilor sale.



CREATE OR REPLACE PROCEDURE raport_vanzari_categorii IS
    -- a) Definim lista „categoriilor premium" (dimensiune fixa => Varray)
    TYPE t_vector_categorii IS VARRAY(3) OF VARCHAR2(50);
    v_categorii_tinta t_vector_categorii := t_vector_categorii('Laptopuri & IT', 'Telefoane Mobile', 'Gaming');

    -- b) Lista ID-urilor clientilor gasiti (nu stim cati vor fi gasiti => Nested Table)
    TYPE t_lista_clienti IS TABLE OF NUMBER;
    v_clienti_gasiti t_lista_clienti := t_lista_clienti();

    -- Asocierea ID_Client -> Suma Totala (Index-By Table)
    TYPE t_statistica_vanzari IS TABLE OF NUMBER INDEX BY BINARY_INTEGER;
    v_totaluri_clienti t_statistica_vanzari;

    -- Variabile auxiliare
    v_total_temp NUMBER;
    v_idx        BINARY_INTEGER; -- Indexul pentru parcurgere
    v_cat_curent VARCHAR2(50);
BEGIN

    -- Populam Nested Table
    FOR i IN 1..v_categorii_tinta.COUNT LOOP
        v_cat_curent := v_categorii_tinta(i);

        FOR r_client IN ( -- selecteaza toti clientii care au cumparat produse din v_cat_curent
```

```oracle_plsql
            SELECT DISTINCT c.id_client
            FROM CLIENTI c, COMENZI com, DETALII_COMANDA dc, PRODUSE p, CATEGORII cat
            WHERE c.id_client = com.id_client
              AND com.id_comanda = dc.id_comanda
              AND dc.id_produs = p.id_produs
              AND p.id_categorie = cat.id_categorie
              AND cat.nume = v_cat_curent
        ) LOOP
            v_clienti_gasiti.EXTEND;
            v_clienti_gasiti(v_clienti_gasiti.LAST) := r_client.id_client;
        END LOOP;
    END LOOP;

    -- Verificam daca am gasit clienti
    IF v_clienti_gasiti.COUNT = 0 THEN
        DBMS_OUTPUT.PUT_LINE( A 'Nu au fost gssiti clienti.');
        RETURN;
    END IF;

    -- Populam Index-By Table
    FOR i IN v_clienti_gasiti.FIRST .. v_clienti_gasiti.LAST LOOP

        SELECT NVL(SUM(dc.cantitate * dc.pret_unitar), 0)
        INTO v_total_temp
        FROM COMENZI com, DETALII_COMANDA dc
        WHERE com.id_comanda = dc.id_comanda
          AND com.id_client = v_clienti_gasiti(i);

        v_totaluri_clienti(v_clienti_gasiti(i)) := v_total_temp;
    END LOOP;

    -- Afisare
    DBMS_OUTPUT.PUT_LINE( A '--- Clientii Premium ---');
```

```
    v_idx := v_totaluri_clienti.FIRST;
    WHILE v_idx IS NOT NULL LOOP
        DBMS_OUTPUT.PUT_LINE( 'Client ID: ' || v_idx ||
                              ' | Valoare Totala Achizitii: ' || v_totaluri_clienti(v_idx) || ' RON');
        v_idx := v_totaluri_clienti.NEXT(v_idx);
    END LOOP;
-- Ordinea clientilor este garantata crescator dupa ID deoarece Index-By Table cu cheie numerica stocheaza elementele sortate automat dupa index
END;
/

BEGIN
    raport_vanzari_categorii;
END;
/
```

```
ONLINE_SHOP> BEGIN
                raport_vanzari_categorii;
            END;
[2026-01-09 16:04:13] completed in 12 ms
--- Clientii Premium ---
Client ID: 1001 | Valoare Totala Achizitii: 14448.98 RON
Client ID: 1003 | Valoare Totala Achizitii: 13400.49 RON
Client ID: 1005 | Valoare Totala Achizitii: 6300 RON
```

7. **Formularea în limbaj natural a unei probleme pe care să o rezolvăm folosind un subprogram stocat independent care să utilizeze 2 tipuri diferite de cursoare studiate, unul dintre acestea fiind cursor parametrizat, dependent de celălalt cursor și apelarea acestuia**

Pentru fiecare categorie de produse, să se afișeze lista produselor aparținând acelei categorii.

a) Se parcurge lista tuturor categoriilor existente.
b) Pentru fiecare categorie găsită, se afișează numele acesteia.
c) Imediat sub nume, se afișează lista produselor aparținând acelei categorii.

```
CREATE OR REPLACE PROCEDURE raport_detaliat_categorii IS

   -- Cursor Explicit Parametrizat
   -- Asteapta un parametru (p_id_cat) pentru a functiona
   CURSOR c_produse (p_id_cat NUMBER) IS
      SELECT denumire, pret, stoc
      FROM PRODUSE
      WHERE id_categorie = p_id_cat
      ORDER BY pret DESC;

   -- Variabile locale pentru a stoca datele din cursorul parametrizat
   v_denumire PRODUSE.denumire%TYPE;
   v_pret     PRODUSE.pret%TYPE;
   v_stoc     PRODUSE.stoc%TYPE;

BEGIN
   -- Ciclu Cursor
   -- Cursorul "Parinte"
   FOR r_cat IN (
      SELECT id_categorie, nume
      FROM CATEGORII
      ORDER BY id_categorie
      ) LOOP

      DBMS_OUTPUT.PUT_LINE('CATEGORIA: ' || r_cat.nume);

      -- Deschidem cursorul parametrizat (Copil), trimitand ID-ul din cursorul Parinte
      OPEN c_produse(r_cat.id_categorie);

      LOOP
         -- Incarcam datele
         FETCH c_produse INTO v_denumire, v_pret, v_stoc;
```

```
        -- Verificam iesirea din bucla
        EXIT WHEN c_produse%NOTFOUND;

        -- Afisam datele produsului
        DBMS_OUTPUT.PUT_LINE('   * Produs: ' || v_denumire ||
                             ' | Pret: ' || v_pret || ' RON' ||
                             ' | Stoc: ' || v_stoc || ' buc.');
      END LOOP;

      -- Verificam daca categoria a avut produse (folosind %ROWCOUNT)
      IF c_produse%ROWCOUNT = 0 THEN
        DBMS_OUTPUT.PUT_LINE('   -> (Nu exista produse asociate acestei categorii)');
      END IF;


      CLOSE c_produse;

    END LOOP;
END;
/


BEGIN
    raport_detaliat_categorii;
END;
/
```

```oracle_plsql
-- Cerinta 7
-- Pentru fiecare categorie de produse, sa se afiseze lista produselor apartinand acelei categorii.
-- a) Se parcurge lista tuturor categoriilor existente.
-- b) Pentru fiecare categorie gasita, se afiseaza numele acesteia.
-- c) Imediat sub nume, se afiseaza lista produselor apartinand acelei categorii.

CREATE OR REPLACE PROCEDURE raport_detaliat_categorii IS

    -- Cursor Explicit Parametrizat
    -- Asteapta un parametru (p_id_cat) pentru a functiona
    CURSOR c_produse (p_id_cat NUMBER) IS
        SELECT denumire, pret, stoc
        FROM PRODUSE
        WHERE id_categorie = p_id_cat
        ORDER BY pret DESC;

    -- Variabile locale pentru a stoca datele din cursorul parametrizat
    v_denumire PRODUSE.denumire%TYPE;
    v_pret     PRODUSE.pret%TYPE;
    v_stoc     PRODUSE.stoc%TYPE;

BEGIN
    -- Ciclu Cursor
    -- Cursorul "Parinte"
    FOR r_cat IN (
        SELECT id_categorie, nume
        FROM CATEGORII
        ORDER BY id_categorie
        ) LOOP

        DBMS_OUTPUT.PUT_LINE( 'CATEGORIA: ' || r_cat.nume);

        -- Deschidem cursorul parametrizat (Copil), trimitand ID-ul din cursorul Parinte
        OPEN c_produse(r_cat.id_categorie);

        LOOP
```

```oracle_plsql
            -- Incarcam datele
            FETCH c_produse INTO v_denumire, v_pret, v_stoc;

            -- Verificam iesirea din bucla
            EXIT WHEN c_produse%NOTFOUND;

            -- Afisam datele produsului
            DBMS_OUTPUT.PUT_LINE( '   * Produs: ' || v_denumire ||
                                  ' | Pret: ' || v_pret || ' RON' ||
                                  ' | Stoc: ' || v_stoc || ' buc.');
        END LOOP;

        -- Verificam daca categoria a avut produse (folosind %ROWCOUNT)
        IF c_produse%ROWCOUNT = 0 THEN
            DBMS_OUTPUT.PUT_LINE( '   -> (Nu exista produse asociate acestei categorii)');
        END IF;


        CLOSE c_produse;

    END LOOP;
END;
/
```

```
706 ✓ BEGIN
707       raport_detaliat_categorii;
708   END;
709   /
710
711
712
```

```
                    raport_detaliat_categorii;
                END;
[2026-01-09 16:07:09] completed in 154 ms
CATEGORIA: Telefoane Mobile
   * Produs: iPhone 15 Pro | Pret: 5400 RON | Stoc: 29 buc.
   * Produs: Samsung Galaxy S23 | Pret: 3999.99 RON | Stoc: 68 buc.
CATEGORIA: Laptopuri & IT
   * Produs: MacBook Air M2 | Pret: 5999 RON | Stoc: 14 buc.
   * Produs: Laptop ASUS TUF Gaming | Pret: 4500.5 RON | Stoc: 29 buc.
   * Produs: Mouse Logitech MX Master | Pret: 450 RON | Stoc: 97 buc.
CATEGORIA: Televizoare & Audio
   * Produs: Televizor LG OLED | Pret: 6200 RON | Stoc: 9 buc.
   * Produs: Soundbar Samsung | Pret: 899 RON | Stoc: 28 buc.
CATEGORIA: Electrocasnice Mici
   * Produs: Espresor Philips LatteGo | Pret: 2100 RON | Stoc: 39 buc.
   * Produs: Aspirator Robot Roborock | Pret: 1850 RON | Stoc: 24 buc.
CATEGORIA: Gaming
   * Produs: Consola PlayStation 5 | Pret: 2699.99 RON | Stoc: 49 buc.
```

8. **Formularea în limbaj natural a unei probleme pe care să o rezolvăm folosind un subprogram stocat independent de tip funcție care să utilizeze într-o singură comandă SQL 3 dintre tabelele create. Vom trata toate excepțiile care pot apărea, incluzând excepțiile predefinite NO_DATA_FOUND și TOO_MANY_ROWS. Vom apela subprogramul astfel încât vom evidenția toate cazurile tratate.**

Identificați numărul facturii emise pentru un anumit client într-o anumită lună.

a) Definiți o funcție care va primi ca parametri numele de familie al clientului și luna/anul de interes.
b) Funcția va returna un mesaj text care să indice fie ID-ul facturii găsite, fie un mesaj de eroare specific în care:
    a. Nu există nicio factură pentru acel client în luna respectivă (NO_DATA_FOUND);
    b. Există mai multe facturi pentru acel client în luna respectivă (clientul a făcut mai multe comenzi în acea lună) -> TOO_MANY_ROWS.

```
CREATE OR REPLACE FUNCTION gaseste_factura_client (
   p_nume_client VARCHAR2,
   p_luna_an     VARCHAR2 -- format asteptat 'MM-YYYY'
)
RETURN VARCHAR2
IS
   v_id_factura FACTURI.id_factura%TYPE;
   v_rezultat   VARCHAR2(200);
BEGIN
   -- Selectam ID-ul facturii. Aceasta comanda uneste 3 tabele
   -- Daca selectul returneaza exact 1 rand, merge mai departe
   -- Daca returneaza 0 sau >1, sare direct in zona de EXCEPTION
   SELECT f.id_factura
   INTO v_id_factura
   FROM FACTURI f
   JOIN COMENZI c ON f.id_comanda = c.id_comanda
   JOIN CLIENTI cl ON c.id_client = cl.id_client
   WHERE UPPER(cl.nume) = UPPER(p_nume_client)
     AND TO_CHAR(c.data_comanda, 'MM-YYYY') = p_luna_an;

   -- S-a gasit exact o factura
   v_rezultat := 'Factura identificata are ID-ul ' || v_id_factura;
   RETURN v_rezultat;

EXCEPTION
   WHEN NO_DATA_FOUND THEN
      RETURN 'Eroare: Nu exista facturi pentru clientul ' || p_nume_client || ' in luna ' || p_luna_an;
```

```
   WHEN TOO_MANY_ROWS THEN
      RETURN 'Eroare: Clientul ' || p_nume_client || ' are mai multe facturi in luna ' || p_luna_an;

   WHEN OTHERS THEN
      RETURN 'Alta eroare: ' || SQLERRM;
END;
/



DECLARE
   v_mesaj VARCHAR2(200);
BEGIN
   -- TOO_MANY_ROWS
   -- Popescu are 2 comenzi in luna 03-2024 (ID 2001 si 2004)
   v_mesaj := gaseste_factura_client('Popescu', '03-2024');
   DBMS_OUTPUT.PUT_LINE('   >> ' || v_mesaj);
   DBMS_OUTPUT.NEW_LINE;

   -- Succes
   -- Radu are o singura comanda in luna 04-2024 (ID 2008)
   v_mesaj := gaseste_factura_client('Radu', '04-2024');
   DBMS_OUTPUT.PUT_LINE('   >> ' || v_mesaj);
   DBMS_OUTPUT.NEW_LINE;

   -- NO_DATA_FOUND
   -- Popescu nu are comenzi in luna 12-2024
   v_mesaj := gaseste_factura_client('Popescu', '12-2024');
   DBMS_OUTPUT.PUT_LINE('   >> ' || v_mesaj);

END;
/
```

```oracle_plsql
--- Cerinta 8
--- Identificati numarul facturii emise pentru un anumit client intr-o anumita luna.
--- a) Definiti o functie care va primi ca parametri numele de familie al clientului si luna/anul de interes.
--- b) Functia va returna un mesaj text care sa indice fie ID-ul facturii gasite, fie un mesaj de eroare specific in care:
---    i) Nu exista nicio factura pentru acel client in luna respectiva (NO_DATA_FOUND)
---    ii) Exista mai multe facturi pentru acel client in luna respectiva (clientul a facut mai multe comenzi in acea luna) -> (TOO_MANY_ROWS).

CREATE OR REPLACE FUNCTION gaseste_factura_client (
    p_nume_client VARCHAR2,
    p_luna_an     VARCHAR2 -- format asteptat 'MM-YYYY'
)
RETURN VARCHAR2
IS
    v_id_factura FACTURI.id_factura%TYPE;
    v_rezultat   VARCHAR2(200);
BEGIN
    -- Selectam ID-ul facturii. Aceasta comanda uneste 3 tabele
    -- Daca selectul returneaza exact 1 rand, merge mai departe
    -- Daca returneaza 0 sau >1, sare direct in zona de EXCEPTION
    SELECT f.id_factura
    INTO v_id_factura
    FROM FACTURI f
    JOIN COMENZI c 1..n<->1: ON f.id_comanda = c.id_comanda
    JOIN CLIENTI cl 1..n<->1: ON c.id_client = cl.id_client
    WHERE UPPER(cl.nume) = UPPER(p_nume_client)
      AND TO_CHAR(c.data_comanda, 'MM-YYYY') = p_luna_an;

    -- S-a gasit exact o factura
    v_rezultat := 'Factura identificata are ID-ul ' || v_id_factura;
    RETURN v_rezultat;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'Eroare: Nu exista facturi pentru clientul ' || p_nume_client || ' in luna ' || p_luna_an;
```

```oracle_plsql

    WHEN TOO_MANY_ROWS THEN
        RETURN 'Eroare: Clientul ' || p_nume_client || ' are mai multe facturi in luna ' || p_luna_an;

    WHEN OTHERS THEN
        RETURN 'Alta eroare: ' || SQLERRM;
END;
/
```

```oracle_plsql
DECLARE
    v_mesaj VARCHAR2(200);
BEGIN
    -- TOO_MANY_ROWS
    -- Popescu are 2 comenzi in luna 03-2024 (ID 2001 si 2004)
    v_mesaj := gaseste_factura_client( p_nume_client 'Popescu', p_luna_an '03-2024');
    DBMS_OUTPUT.PUT_LINE( A '   >> ' || v_mesaj);
    DBMS_OUTPUT.NEW_LINE;

    -- Succes
    -- Radu are o singura comanda in luna 04-2024 (ID 2008)
    v_mesaj := gaseste_factura_client( p_nume_client 'Radu', p_luna_an '04-2024');
    DBMS_OUTPUT.PUT_LINE( A '   >> ' || v_mesaj);
    DBMS_OUTPUT.NEW_LINE;

    -- NO_DATA_FOUND
    -- Popescu nu are comenzi in luna 12-2024
    v_mesaj := gaseste_factura_client( p_nume_client 'Popescu', p_luna_an '12-2024');
    DBMS_OUTPUT.PUT_LINE( A '   >> ' || v_mesaj);

END;
/
```

```oracle_plsql
                -- NO_DATA_FOUND
                -- Popescu nu are comenzi in luna 12-2024
                v_mesaj := gaseste_factura_client('Popescu', '12-2024');
                DBMS_OUTPUT.PUT_LINE('   >> ' || v_mesaj);

            END;
[2026-01-09 16:09:41] completed in 26 ms
   >> Eroare: Clientul Popescu are mai multe facturi in luna 03-2024
   >> Factura identificata are ID-ul 3008
   >> Eroare: Nu exista facturi pentru clientul Popescu in luna 12-2024
```

9. **Formularea în limbaj natural a unei probleme pe care să o rezolvăm folosind un subprogram stocat independent de tip procedură care să aibă minimum 2 parametri și să utilizeze într-o singură comandă SQL 5 dintre tabelele create. Vom defini minimum 2 excepții proprii, altele decât cele predefinite la nivel de sistem. Vom apela subprogramul astfel încât vom evidenția toate cazurile definite și tratate.**

Verificați dacă un client (specificat prin nume) este eligibil pentru un „Bonus de fidelitate" acordat pentru achizițiile dintr-o anumită categorie de produse (specificată prin nume). Un client este eligibil pentru acordarea bonusului dacă valoarea totală a produselor cumpărate de acel client din categoria respectivă depășește 2000 RON.

a) Calculați valoarea totală a produselor cumpărate de acel client din categoria respectivă.
b) Tratați următoarele excepții:
    a. ex_fara_achizitii – În cazul în care clientul nu a cumpărat niciun produs din categoria specificată (suma este 0 sau NULL);
    b. ex_bonus_neeligibil – Dacă clientul a cumpărat produse, dar valoarea totală este sub pragul minim de 2000 RON necesar pentru acordarea bonusului.

```
CREATE OR REPLACE PROCEDURE verificare_eligibilitate_bonus (
   p_nume_client    IN VARCHAR2,
   p_nume_categorie IN VARCHAR2
) IS
   -- Variabila pentru stocarea sumei
   v_total_achizitii NUMBER := 0;

   -- Constanta pentru pragul de bonus
   c_prag_bonus      CONSTANT NUMBER := 2000;

   -- Definirea exceptiilor proprii
   ex_fara_achizitii   EXCEPTION;
   ex_bonus_neeligibil EXCEPTION;

BEGIN
   -- Comanda  pe 5 tabele
   -- Folosim NVL pentru a transforma NULL in 0 daca nu exista potriviri
   SELECT NVL(SUM(dc.cantitate * dc.pret_unitar), 0)
   INTO v_total_achizitii
   FROM CLIENTI c
   JOIN COMENZI com ON c.id_client = com.id_client
   JOIN DETALII_COMANDA dc ON com.id_comanda = dc.id_comanda
   JOIN PRODUSE p ON dc.id_produs = p.id_produs
   JOIN CATEGORII cat ON p.id_categorie = cat.id_categorie
   WHERE UPPER(c.nume) = UPPER(p_nume_client)
     AND UPPER(cat.nume) = UPPER(p_nume_categorie);
```

```
   -- Nu a cumparat nimic din acea categorie
   IF v_total_achizitii = 0 THEN
      RAISE ex_fara_achizitii;
   END IF;

   -- A cumparat, dar suma e prea mica
   IF v_total_achizitii < c_prag_bonus THEN
      RAISE ex_bonus_neeligibil;
   END IF;

   -- Succes
   DBMS_OUTPUT.PUT_LINE('Clientul ' || p_nume_client ||
                  ' este eligibil pentru bonus! Total achizitii in categoria ' ||
                  p_nume_categorie || ': ' || v_total_achizitii || ' RON.');

EXCEPTION
   WHEN ex_fara_achizitii THEN
      DBMS_OUTPUT.PUT_LINE('Clientul ' || p_nume_client ||
                     ' nu a efectuat nicio achizitie in categoria ' || p_nume_categorie || '.');

   WHEN ex_bonus_neeligibil THEN
      DBMS_OUTPUT.PUT_LINE('Clientul ' || p_nume_client ||
                     ' are achizitii de doar ' || v_total_achizitii ||
                     ' RON in categoria ' || p_nume_categorie ||
                     '. Pragul necesar este ' || c_prag_bonus || ' RON.');

   -- Tratarea erorilor neprevazute
   WHEN OTHERS THEN
      DBMS_OUTPUT.PUT_LINE('EROARE: A aparut o problema tehnica: ' || SQLERRM);
END;
/



BEGIN
   -- Succes
   verificare_eligibilitate_bonus('Popescu', 'Telefoane Mobile');
   DBMS_OUTPUT.NEW_LINE;

   -- Fara achizitii
   verificare_eligibilitate_bonus('Popescu', 'Electrocasnice Mici');
   DBMS_OUTPUT.NEW_LINE;

   -- Suma mica
```

verificare_eligibilitate_bonus('Ionescu', 'Televizoare & Audio');
END;
/

```oracle
-- Cerinta 9
-- Verificati daca un client (specificat prin nume) este eligibil pentru un „Bonus de fidelitate" acordat pentru achizitiile dintr-o anumita categorie de produse (specificata prin nume). Un
-- client este eligibil pentru acordarea bonusului daca valoarea totala a produselor cumparate de acel client din categoria respectiva depaseste 2000 RON.
-- a) Calculati valoarea totala a produselor cumparate de acel client din categoria respectiva.
-- b) Tratati urmatoarele exceptii:
--    i) ex_fara_achizitii - In cazul in care clientul nu a cumparat niciun produs din categoria specificata (suma este 0 sau NULL)
--    ii) ex_bonus_neeligibil - Daca clientul a cumparat produse, dar valoarea totala este sub pragul minim de 2000 RON necesar pentru acordarea bonusului.

CREATE OR REPLACE PROCEDURE verificare_eligibilitate_bonus (
    p_nume_client    IN VARCHAR2,
    p_nume_categorie IN VARCHAR2
) IS
    -- Variabila pentru stocarea sumei
    v_total_achizitii NUMBER := 0;

    -- Constanta pentru pragul de bonus
    c_prag_bonus      CONSTANT NUMBER := 2000;

    -- Definirea exceptiilor proprii
    ex_fara_achizitii   EXCEPTION;
    ex_bonus_neeligibil EXCEPTION;

BEGIN
    -- Comanda  pe 5 tabele
    -- Folosim NVL pentru a transforma NULL in 0 daca nu exista potriviri
    SELECT NVL(SUM(dc.cantitate * dc.pret_unitar), 0)
    INTO v_total_achizitii
    FROM CLIENTI c
    JOIN COMENZI com 1<->1..n: ON c.id_client = com.id_client
```

```
END;
[2026-01-09 16:12:17] completed in 158 ms
Clientul Popescu este eligibil pentru bonus! Total achizitii in categoria Telefoane Mobile: 7999.98 RON.
Clientul Popescu nu a efectuat nicio achizitie in categoria Electrocasnice Mici.
Clientul Ionescu are achizitii de doar 1798 RON in categoria Televizoare & Audio. Pragul necesar este 2000 RON.
```

```oracle
    JOIN DETALII_COMANDA dc 1<->1..n: ON com.id_comanda = dc.id_comanda
    JOIN PRODUSE p 1..n<->1: ON dc.id_produs = p.id_produs
    JOIN CATEGORII cat 1..n<->1: ON p.id_categorie = cat.id_categorie
    WHERE UPPER(c.nume) = UPPER(p_nume_client)
      AND UPPER(cat.nume) = UPPER(p_nume_categorie);



    -- Nu a cumparat nimic din acea categorie
    IF v_total_achizitii = 0 THEN
        RAISE ex_fara_achizitii;
    END IF;

    -- A cumparat, dar suma e prea mica
    IF v_total_achizitii < c_prag_bonus THEN
        RAISE ex_bonus_neeligibil;
    END IF;

    -- Succes
    DBMS_OUTPUT.PUT_LINE( A 'Clientul ' || p_nume_client ||
                          ' este eligibil pentru bonus! Total achizitii in categoria ' ||
                          p_nume_categorie || ': ' || v_total_achizitii || ' RON.');

EXCEPTION
    WHEN ex_fara_achizitii THEN
        DBMS_OUTPUT.PUT_LINE( A 'Clientul ' || p_nume_client ||
                              ' nu a efectuat nicio achizitie in categoria ' || p_nume_categorie || '.');
```

```
END;
[2026-01-09 16:12:17] completed in 158 ms
Clientul Popescu este eligibil pentru bonus! Total achizitii in categoria Telefoane Mobile: 7999.98 RON.
Clientul Popescu nu a efectuat nicio achizitie in categoria Electrocasnice Mici.
Clientul Ionescu are achizitii de doar 1798 RON in categoria Televizoare & Audio. Pragul necesar este 2000 RON.
```

```oracle_plsql
    WHEN ex_bonus_neeligibil THEN
        DBMS_OUTPUT.PUT_LINE( 'Clientul ' || p_nume_client ||
                              ' are achizitii de doar ' || v_total_achizitii ||
                              ' RON in categoria ' || p_nume_categorie ||
                              '. Pragul necesar este ' || c_prag_bonus || ' RON.');

    -- Tratarea erorilor neprevazute
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE( 'EROARE: A aparut o problema tehnica: ' || SQLERRM);
END;
/



BEGIN
    -- Succes
    verificare_eligibilitate_bonus( p_nume_client 'Popescu', p_nume_categorie 'Telefoane Mobile');
    DBMS_OUTPUT.NEW_LINE;

    -- Fara achizitii
    verificare_eligibilitate_bonus( p_nume_client 'Popescu', p_nume_categorie 'Electrocasnice Mici');
    DBMS_OUTPUT.NEW_LINE;

    -- Suma mica
    verificare_eligibilitate_bonus( p_nume_client 'Ionescu', p_nume_categorie 'Televizoare & Audio');
END;
/
```

```
END;
[2026-01-09 16:12:17] completed in 158 ms
Clientul Popescu este eligibil pentru bonus! Total achizitii in categoria Telefoane Mobile: 7999.98 RON.
Clientul Popescu nu a efectuat nicio achizitie in categoria Electrocasnice Mici.
Clientul Ionescu are achizitii de doar 1798 RON in categoria Televizoare & Audio. Pragul necesar este 2000 RON.
```

10. Definirea unui trigger de tip LMD la nivel de comandă și declanșarea acestuia

Definim un trigger LMD la nivel de comandă care să monitorizeze activitatea asupra tabelului PRODUSE, astfel încât după executarea oricărei comenzi de INSERT, UPDATE sau DELETE asupra acestui tabel să se afișeze un mesaj informativ care să precizeze tipul operației efectuate și data/ora la care a avut loc.

```
CREATE OR REPLACE TRIGGER trg_audit_operatii_produse
   AFTER INSERT OR UPDATE OR DELETE ON PRODUSE
DECLARE
   v_tip_operatie VARCHAR2(20);
BEGIN
   IF INSERTING THEN
      v_tip_operatie := 'INSERARE';
   ELSIF UPDATING THEN
      v_tip_operatie := 'ACTUALIZARE';
   ELSIF DELETING THEN
      v_tip_operatie := 'STERGERE';
   END IF;

   DBMS_OUTPUT.PUT_LINE('S-a executat o operatie de ' || v_tip_operatie ||
                  ' asupra tabelului PRODUSE la data de ' ||
                  TO_CHAR(SYSDATE, 'DD-MM-YYYY HH24:MI:SS'));
END;
/

BEGIN
   -- Declansam trigger-ul printr-un UPDATE
   -- Vom scumpi produsele din categoria 1 cu 1 RON
   UPDATE PRODUSE
   SET pret = pret + 1
   WHERE id_categorie = 1;

   ROLLBACK;
END;
/
```

```oracle_plsql
-- Cerinta 10
-- Definim un trigger LMD la nivel de comanda care sa monitorizeze activitatea asupra tabelului PRODUSE, astfel incat dupa executarea oricarei comenzi de
-- INSERT, UPDATE sau DELETE asupra acestui tabel sa se afiseze un mesaj informativ care sa precizeze tipul operatiei efectuate
-- si data/ora la care a avut loc.

CREATE OR REPLACE TRIGGER trg_audit_operatii_produse
    AFTER INSERT OR UPDATE OR DELETE ON PRODUSE
DECLARE
    v_tip_operatie VARCHAR2(20);
BEGIN
    IF INSERTING THEN
        v_tip_operatie := 'INSERARE';
    ELSIF UPDATING THEN
        v_tip_operatie := 'ACTUALIZARE';
    ELSIF DELETING THEN
        v_tip_operatie := 'STERGERE';
    END IF;

    DBMS_OUTPUT.PUT_LINE( A 'S-a executat o operatie de ' || v_tip_operatie ||
                          ' asupra tabelului PRODUSE la data de ' ||
                          TO_CHAR(SYSDATE, 'DD-MM-YYYY HH24:MI:SS'));
END;
/

BEGIN
    -- Declansam trigger-ul printr-un UPDATE
    -- Vom scumpi produsele din categoria 1 cu 1 RON
    UPDATE PRODUSE
    SET pret = pret + 1
    WHERE id_categorie = 1;
```

```
END;
[2026-01-09 16:17:49] completed in 53 ms
S-a executat o operatie de ACTUALIZARE asupra tabelului PRODUSE la data de 09-01-2026 16:17:49
```

```oracle_plsql
    ROLLBACK;
END;
/
```

```
END;
[2026-01-09 16:17:49] completed in 53 ms
S-a executat o operatie de ACTUALIZARE asupra tabelului PRODUSE la data de 09-01-2026 16:17:49
```

11. Definirea unui trigger de tip LMD la nivel de linie și declanșarea acestuia

Triggeri care se încadrează în această categorie sunt deja definiți la crearea tabelelor (ex. **trg_gestiune_stoc_vanzari**, **trg_gestiune_stoc_furnizor, trg_check_data_angajare** etc.).

Vom lua ca exemplu trigger-ul **trg_gestiune_stoc_vanzari**, definit pentru actualizarea stocului din PRODUSE și declanșat de INSERT-urile / UPDATE-urile / DELETE-urile din DETALII_COMANDA.

```
CREATE OR REPLACE TRIGGER trg_gestiune_stoc_vanzari
BEFORE INSERT OR UPDATE OR DELETE ON DETALII_COMANDA
FOR EACH ROW
FOLLOWS trg_blocare_detalii_livrate
DECLARE
   v_stoc_curent INT;
   v_id_produs   INT;
BEGIN
   -- Identificam ID-ul produsului (cel nou la insert/update, cel vechi la delete)
   v_id_produs := CASE WHEN DELETING THEN :OLD.id_produs ELSE :NEW.id_produs END;

   -- Selectam stocul curent cu blocare (FOR UPDATE - blocam randul in situatia in care doi clienti
comanda ultimul produs in acelasi timp)
   BEGIN
      SELECT stoc INTO v_stoc_curent
      FROM PRODUSE
      WHERE id_produs = v_id_produs
      FOR UPDATE;
   EXCEPTION
      WHEN NO_DATA_FOUND THEN
         RAISE_APPLICATION_ERROR(-20006, 'Produsul cu ID ' || v_id_produs || ' nu a fost gasit!');
   END;

   IF INSERTING THEN
      -- Verificam daca avem destul stoc pentru noua vanzare
      IF v_stoc_curent < :NEW.cantitate THEN
         RAISE_APPLICATION_ERROR(-20005, 'Stoc insuficient! Disponibil: ' || v_stoc_curent);
      END IF;

      UPDATE PRODUSE SET stoc = stoc - :NEW.cantitate WHERE id_produs = v_id_produs;

   ELSIF UPDATING THEN
      -- Verificam daca noul stoc (stoc_curent + ce era deja rezervat - ce se cere acum) este suficient
      -- Ex. - Avem 10 in stoc, am comandat 5 (deci in total erau 15). Modificam la 12
```

```
        -- Verificam daca 15 >= 12
        IF (v_stoc_curent + :OLD.cantitate) < :NEW.cantitate THEN
            RAISE_APPLICATION_ERROR(-20005, 'Stoc insuficient pentru modificare! Maxim
disponibil: ' || (v_stoc_curent + :OLD.cantitate));
        END IF;

        UPDATE PRODUSE SET stoc = stoc + (:OLD.cantitate - :NEW.cantitate) WHERE id_produs =
v_id_produs;

    ELSIF DELETING THEN
        -- Punem produsele inapoi in stoc daca se sterge comanda
        UPDATE PRODUSE SET stoc = stoc + :OLD.cantitate WHERE id_produs = v_id_produs;
    END IF;
END;
/
```

```
-- Trigger pentru actualizarea stocului -- cerinta 11, declansat de INSERT-urile / UPDATE-urile / DELETE-urile din DETALII_COMANDA
CREATE OR REPLACE TRIGGER trg_gestiune_stoc_vanzari
BEFORE INSERT OR UPDATE OR DELETE ON DETALII_COMANDA
FOR EACH ROW
FOLLOWS trg_blocare_detalii_livrate
DECLARE
    v_stoc_curent INT;
    v_id_produs   INT;
BEGIN
    -- Identificam ID-ul produsului (cel nou la insert/update, cel vechi la delete)
    v_id_produs := CASE WHEN DELETING THEN :OLD.id_produs ELSE :NEW.id_produs END;

    -- Selectam stocul curent cu blocare (FOR UPDATE - blocam randul in situatia in care doi clienti comanda ultimul produs in acelasi timp)
    BEGIN
        SELECT stoc INTO v_stoc_curent
        FROM PRODUSE
        WHERE id_produs = v_id_produs
        FOR UPDATE;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20006, 'Produsul cu ID ' || v_id_produs || ' nu a fost gasit!');
    END;

    IF INSERTING THEN
        -- Verificam daca avem destul stoc pentru noua vanzare
        IF v_stoc_curent < :NEW.cantitate THEN
            RAISE_APPLICATION_ERROR(-20005, 'Stoc insuficient! Disponibil: ' || v_stoc_curent);
        END IF;

        UPDATE PRODUSE SET stoc = stoc - :NEW.cantitate WHERE id_produs = v_id_produs;

    ELSIF UPDATING THEN
        -- Verificam daca noul stoc (stoc_curent + ce era deja rezervat - ce se cere acum) este suficient
        -- Ex. - Avem 10 in stoc, am comandat 5 (deci in total erau 15). Modificam la 12
```

```
        -- Verificam daca 15 >= 12
        IF (v_stoc_curent + :OLD.cantitate) < :NEW.cantitate THEN
            RAISE_APPLICATION_ERROR(-20005, 'Stoc insuficient pentru modificare! Maxim disponibil: ' || (v_stoc_curent + :OLD.cantitate));
        END IF;

        UPDATE PRODUSE SET stoc = stoc + (:OLD.cantitate - :NEW.cantitate) WHERE id_produs = v_id_produs;

    ELSIF DELETING THEN
        -- Punem produsele inapoi in stoc daca se sterge comanda
        UPDATE PRODUSE SET stoc = stoc + :OLD.cantitate WHERE id_produs = v_id_produs;
    END IF;
END;
/
```

console_1 [@fabianmiu15] console [@localhost] console [db1_grupa241] console [Magazin Online] PRODUSE [Magazin Online]

Tx: Manual DDL

WHERE ORDER BY

	ID_PRODUS	ID_CATEGORIE	DENUMIRE	PRET	STOC
1	101	1	Samsung Galaxy S23	3999.99	68
2	102	1	iPhone 15 Pro	5400.00	29
3	103	2	Laptop ASUS TUF Gaming	4500.50	29
4	104	2	MacBook Air M2	5999.00	14
5	105	3	Televizor LG OLED	6200.00	9
6	106	4	Espresor Philips LatteGo	2100.00	39
7	107	4	Aspirator Robot Roborock	1850.00	24
8	108	5	Consola PlayStation 5	2699.99	49
9	109	2	Mouse Logitech MX Master	450.00	97
10	110	3	Soundbar Samsung	899.00	28

12. Definirea unui trigger de tip LDD și declanșarea acestuia

Vom defini un trigger care să protejeze baza de date, blocând operația de ștergere a unui tabel.

```
CREATE OR REPLACE TRIGGER trg_blocare_drop_tabel
   BEFORE DROP ON SCHEMA
BEGIN
   IF ORA_DICT_OBJ_TYPE = 'TABLE' THEN
      RAISE_APPLICATION_ERROR(-20900, 'Nu aveti permisiunea sa stergeti tabelul ' ||
                              ORA_DICT_OBJ_NAME || '. Dezactivati trigger-ul
trg_blocare_drop_tabel.');
   END IF;
END;
/


BEGIN
   EXECUTE IMMEDIATE 'CREATE TABLE tabel_test_ldd (id NUMBER)';
   DBMS_OUTPUT.NEW_LINE;
   EXECUTE IMMEDIATE 'DROP TABLE tabel_test_ldd';
END;
/
```

```oracle_plsql
-- Cerinta 12
-- Definim un trigger care sa protejeze baza de date, blocand operatia de stergere a unui tabel.

CREATE OR REPLACE TRIGGER trg_blocare_drop_tabel
    BEFORE DROP ON SCHEMA
BEGIN
    IF ORA_DICT_OBJ_TYPE = 'TABLE' THEN
        RAISE_APPLICATION_ERROR(-20900, 'Nu aveti permisiunea sa stergeti tabelul ' ||
                                        ORA_DICT_OBJ_NAME || '. Dezactivati trigger-ul trg_blocare_drop_tabel.');
    END IF;
END;
/


BEGIN
    EXECUTE IMMEDIATE 'CREATE TABLE tabel_test_ldd (id NUMBER)';
    DBMS_OUTPUT.NEW_LINE;
    EXECUTE IMMEDIATE 'DROP TABLE tabel_test_ldd';
END;
/

DROP TRIGGER trg_blocare_drop_tabel;
```

```
[42000][4088]
ORA-04088: error during execution of trigger 'ONLINE_SHOP.TRG_BLOCARE_DROP_TABEL'
ORA-00604: error occurred at recursive SQL level 2
ORA-20900: Nu aveti permisiunea sa stergeti tabelul TABEL_TEST_LDD. Dezactivati trigger-ul trg_blocare_drop_tabel.
ORA-06512: at lin ...
```

```
                END;
[2026-01-09 16:27:25] [42000][4088]
[2026-01-09 16:27:25]   ORA-04088: error during execution of trigger 'ONLINE_SHOP.TRG_BLOCARE_DROP_TABEL'
[2026-01-09 16:27:25]   ORA-00604: error occurred at recursive SQL level 2
[2026-01-09 16:27:25]   ORA-20900: Nu aveti permisiunea sa stergeti tabelul TABEL_TEST_LDD. Dezactivati trigger-ul trg_blocare_drop_tabel.
[2026-01-09 16:27:25]   ORA-06512: at line 3
[2026-01-09 16:27:25]   ORA-06512: at line 4
[2026-01-09 16:27:25] Position: 0
```

13. **Formularea în limbaj natural a unei probleme pe care să o rezolvăm folosind un pachet care să includă tipuri de date complexe și obiecte necesare unui flux de acțiuni integrate, specifice bazei de date definite (minimum 2 tipuri de date, minimum 2 funcții, minimum 2 proceduri)**

Definiți un pachet care să gestioneze un coș de cumpărături. Pachetul trebuie să permită adăugarea produselor în coș pe baza ID-ului acestora, preluarea automată a numelui și a prețului produsului din PRODUSE, precum și afișarea bonului fiscal cu lista produselor adăugate și valoarea de plată. Coșul de cumpărături se va goli după afișare.

```
CREATE OR REPLACE PACKAGE pachet_cos_cumparaturi AS

   -- Tip Record (Structura unui produs in cos)
   TYPE r_produs IS RECORD (
      nume_p VARCHAR2(100),
      pret_p NUMBER
   );

   -- Tip Nested Table (Lista de produse)
   TYPE t_lista_produse IS TABLE OF r_produs;

   -- Cosul de cumparaturi
   g_cos t_lista_produse := t_lista_produse();

   -- Functii
   FUNCTION get_pret_produs(p_id NUMBER) RETURN NUMBER;
   FUNCTION get_nume_produs(p_id NUMBER) RETURN VARCHAR2;

   -- Proceduri
   PROCEDURE adauga_produs(p_id NUMBER);
   PROCEDURE afiseaza_bon_fiscal;

END pachet_cos_cumparaturi;
/

CREATE OR REPLACE PACKAGE BODY pachet_cos_cumparaturi AS

   -- Functia 1 - Cauta pretul in baza de date
   FUNCTION get_pret_produs(p_id NUMBER) RETURN NUMBER IS
      v_pret NUMBER;
   BEGIN
      SELECT pret INTO v_pret FROM PRODUSE WHERE id_produs = p_id;
      RETURN v_pret;
   EXCEPTION
```

```
   WHEN NO_DATA_FOUND THEN RETURN 0;
END get_pret_produs;

-- Functia 2 - Cauta numele in baza de date
FUNCTION get_nume_produs(p_id NUMBER) RETURN VARCHAR2 IS
   v_nume VARCHAR2(100);
BEGIN
   SELECT denumire INTO v_nume FROM PRODUSE WHERE id_produs = p_id;
   RETURN v_nume;
EXCEPTION
   WHEN NO_DATA_FOUND THEN RETURN 'Produs Inexistent';
END get_nume_produs;

-- Procedura 1 - Adauga in lista
PROCEDURE adauga_produs(p_id NUMBER) IS
   v_produs_nou r_produs; -- Variabila de tipul Record
BEGIN
   -- Construim produsul
   v_produs_nou.nume_p := get_nume_produs(p_id);
   v_produs_nou.pret_p := get_pret_produs(p_id);

   -- Il adaugam in cos
   g_cos.EXTEND;
   g_cos(g_cos.LAST) := v_produs_nou;

   DBMS_OUTPUT.PUT_LINE('Adaugat in cos: ' || v_produs_nou.nume_p);
END adauga_produs;

-- Procedura 2 - Afiseaza tot ce am strans in lista
PROCEDURE afiseaza_bon_fiscal IS
   v_total NUMBER := 0;
BEGIN
   DBMS_OUTPUT.NEW_LINE;
   DBMS_OUTPUT.PUT_LINE('BONUL FISCAL');

   IF g_cos.COUNT > 0 THEN
      FOR i IN g_cos.FIRST .. g_cos.LAST LOOP
         DBMS_OUTPUT.PUT_LINE(i || '. ' || g_cos(i).nume_p || ' ..... ' || g_cos(i).pret_p || ' RON');
         v_total := v_total + g_cos(i).pret_p;
      END LOOP;

      DBMS_OUTPUT.PUT_LINE('TOTAL DE PLATA: ' || v_total || ' RON');
   ELSE
      DBMS_OUTPUT.PUT_LINE('Cosul este gol.');
   END IF;
```

```
    g_cos.DELETE;
  END afiseaza_bon_fiscal;

END pachet_cos_cumparaturi;
/



BEGIN
  pachet_cos_cumparaturi.adauga_produs(101); -- Samsung S23
  pachet_cos_cumparaturi.adauga_produs(109); -- Mouse
  pachet_cos_cumparaturi.adauga_produs(105); -- Televizor LG

  pachet_cos_cumparaturi.afiseaza_bon_fiscal;
END;
/
```

```
936     -- Cerinta 13
937     -- Definiti un pachet care sa gestioneze un cos de cumparaturi. Pachetul trebuie sa permita adaugarea produselor in cos pe baza ID-ului acestora,
938     -- preluarea automata a numelui si a pretului produsului din PRODUSE, precum si afisarea bonului fiscal cu lista produselor adaugate si
939     -- valoarea de plata. Cosul de cumparaturi se va goli dupa afisare.
940
941     CREATE OR REPLACE PACKAGE pachet_cos_cumparaturi AS
942
943         -- Tip Record (Structura unui produs in cos)
944         TYPE r_produs IS RECORD (
945             nume_p VARCHAR2(100),
946             pret_p NUMBER
947         );
948
949         -- Tip Nested Table (Lista de produse)
950         TYPE t_lista_produse IS TABLE OF r_produs;
951
952         -- Cosul de cumparaturi
953         g_cos t_lista_produse := t_lista_produse();
954
955         -- Functii
956         FUNCTION get_pret_produs(p_id NUMBER) RETURN NUMBER;
957         FUNCTION get_nume_produs(p_id NUMBER) RETURN VARCHAR2;
958
959         -- Proceduri
960         PROCEDURE adauga_produs(p_id NUMBER);
961         PROCEDURE afiseaza_bon_fiscal;
962
963     END pachet_cos_cumparaturi;
964     /
965
966     CREATE OR REPLACE PACKAGE BODY pachet_cos_cumparaturi AS
967
```

```oracle_plsql
968     -- Functia 1 - Cauta pretul in baza de date
969     FUNCTION get_pret_produs(p_id NUMBER) RETURN NUMBER IS
970         v_pret NUMBER;
971     BEGIN
972         SELECT pret INTO v_pret FROM PRODUSE WHERE id_produs = p_id;
973         RETURN v_pret;
974     EXCEPTION
975         WHEN NO_DATA_FOUND THEN RETURN 0;
976     END get_pret_produs;
977
978     -- Functia 2 - Cauta numele in baza de date
979     FUNCTION get_nume_produs(p_id NUMBER) RETURN VARCHAR2 IS
980         v_nume VARCHAR2(100);
981     BEGIN
982         SELECT denumire INTO v_nume FROM PRODUSE WHERE id_produs = p_id;
983         RETURN v_nume;
984     EXCEPTION
985         WHEN NO_DATA_FOUND THEN RETURN 'Produs Inexistent';
986     END get_nume_produs;
987
988     -- Procedura 1 - Adauga in lista
989     PROCEDURE adauga_produs(p_id NUMBER) IS
990         v_produs_nou r_produs; -- Variabila de tipul Record
991     BEGIN
992         -- Construim produsul
993         v_produs_nou.nume_p := get_nume_produs( p_id p_id);
994         v_produs_nou.pret_p := get_pret_produs( p_id p_id);
995
996         -- Il adaugam in cos
997         g_cos.EXTEND;
998         g_cos(g_cos.LAST) := v_produs_nou;
999
1000        DBMS_OUTPUT.PUT_LINE( A 'Adaugat in cos: ' || v_produs_nou.nume_p);
```

```oracle_plsql
1001    END adauga_produs;
1002
1003    -- Procedura 2 - Afiseaza tot ce am strans in lista
1004    PROCEDURE afiseaza_bon_fiscal IS
1005        v_total NUMBER := 0;
1006    BEGIN
1007        DBMS_OUTPUT.NEW_LINE;
1008        DBMS_OUTPUT.PUT_LINE( A 'BONUL FISCAL');
1009
1010        IF g_cos.COUNT > 0 THEN
1011            FOR i IN g_cos.FIRST .. g_cos.LAST LOOP
1012                DBMS_OUTPUT.PUT_LINE( A i || '. ' || g_cos(i).nume_p || ' ..... ' || g_cos(i).pret_p || ' RON');
1013                v_total := v_total + g_cos(i).pret_p;
1014            END LOOP;
1015
1016            DBMS_OUTPUT.PUT_LINE( A 'TOTAL DE PLATA: ' || v_total || ' RON');
1017        ELSE
1018            DBMS_OUTPUT.PUT_LINE( A 'Cosul este gol.');
1019        END IF;
1020
```

```
        g_cos.DELETE;
    END afiseaza_bon_fiscal;

END pachet_cos_cumparaturi;
/



BEGIN
    pachet_cos_cumparaturi.adauga_produs( P_ID 101); -- Samsung S23
    pachet_cos_cumparaturi.adauga_produs( P_ID 109); -- Mouse
    pachet_cos_cumparaturi.adauga_produs( P_ID 105); -- Televizor LG

    pachet_cos_cumparaturi.afiseaza_bon_fiscal;
END;
/
```

Output | SYS.USER_TRIGGERS

```
                pachet_cos_cumparaturi.afiseaza_bon_fiscal;
            END;
[2026-01-09 16:29:42] completed in 46 ms
Adaugat in cos: Samsung Galaxy S23
Adaugat in cos: Mouse Logitech MX Master
Adaugat in cos: Televizor LG OLED
BONUL FISCAL
1. Samsung Galaxy S23 ..... 3999.99 RON
2. Mouse Logitech MX Master ..... 450 RON
3. Televizor LG OLED ..... 6200 RON
```